Kubernetes

Szkolenie Docker + Kubernetes · część 2 · ALX

Plan części Kubernetes

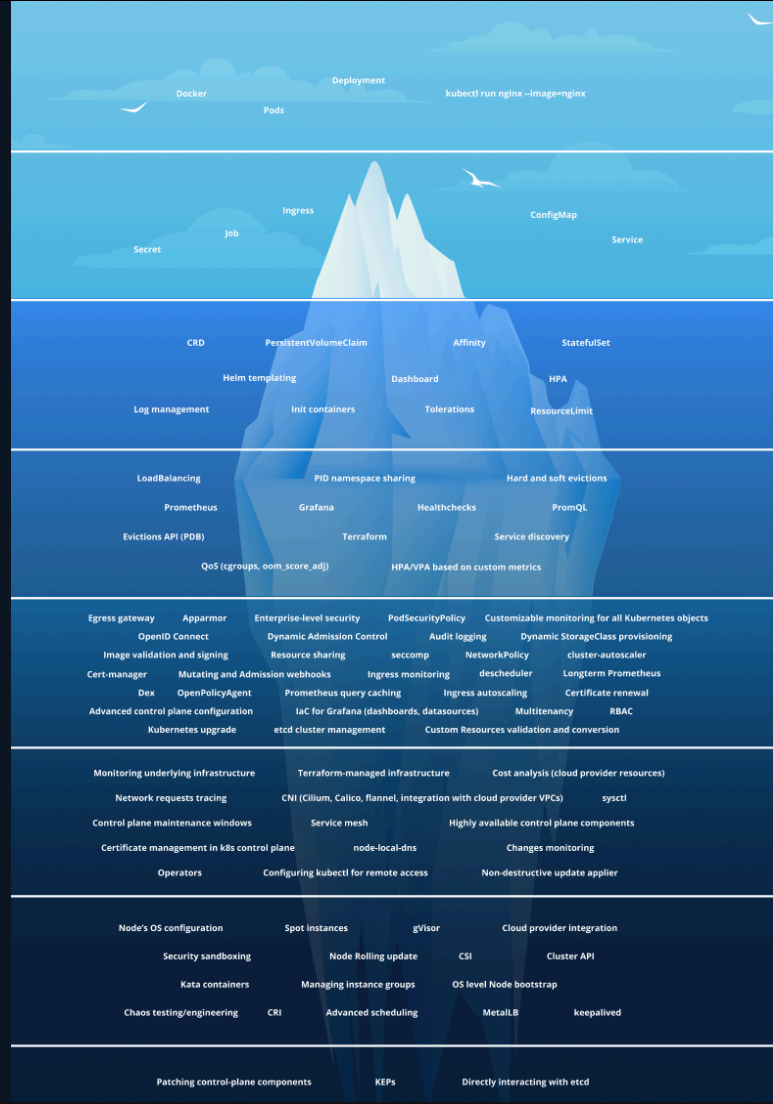
Dzień 2	Sekcja
13:45–14:00	start, środowisko
14:00–14:45	1. Wprowadzenie
14:45–15:25	2. Instalacja
15:40–17:00	3. Podstawowe operacje

Dzień 3	Sekcja
9:00–10:30	4. Networking
10:45–12:00	5. Storage
12:00–13:00	6. Zaawansowane
13:45–14:45	7. kubeadm na VM
15:00–15:30	8. RBAC
15:30–16:10	9. Monitoring
16:10–16:40	10. Troubleshooting
16:40–17:00	11. AI (dodatek) · koniec

★ **przy tytule slajdu – materiał dodatkowy**

1. Wprowadzenie do Kubernetesa

historia · możliwości · architektura: Control Plane, węzły, Pod


Docker
Deployment
kubectl run nginx --image=nginx
Pods
Ingress
ConfigMap
Job
Service
Secret
CRD
PersistentVolumeClaim
Affinity
StatefulSet
Helm templating
Dashboard
HPA
Log management
Init containers
Tolerations
ResourceLimit
LoadBalancing
PID namespace sharing
Hard and soft evictions
Prometheus
Grafana
Healthchecks
PromQL
Evictions API (PDB)
Terraform
Service discovery
QoS (cgroups, oom_score_adj)
HPA/VPA based on custom metrics
Egress gateway
Apparmor
Enterprise-level security
PodSecurityPolicy
Customizable monitoring for all Kubernetes objects
OpenID Connect
Dynamic Admission Control
Audit logging
Dynamic StorageClass provisioning
Image validation and signing
Resource sharing
seccomp
NetworkPolicy
cluster-autoscaler
Cert-manager
Mutating and Admission webhooks
Ingress monitoring
descheduler
Longterm Prometheus
Dex
OpenPolicyAgent
Prometheus query caching
Ingress autoscaling
Certificate renewal
Advanced control plane configuration
IaC for Grafana (dashboards, datasources)
Multitenancy
RBAC
Kubernetes upgrade
etcd cluster management
Custom Resources validation and conversion
Monitoring underlying infrastructure
Terraform-managed infrastructure
Cost analysis (cloud provider resources)
Network requests tracing
CNI (Cilium, Calico, flannel, integration with cloud provider VPCs)
sysctl
Control plane maintenance windows
Service mesh
Highly available control plane components
Certificate management in k8s control plane
node-local-dns
Changes monitoring
Operators
Configuring kubectl for remote access
Non-destructive update applier
Node's OS configuration
Spot instances
gVisor
Cloud provider integration
Security sandboxing
Node Rolling update
CSI
Cluster API
Kata containers
Managing instance groups
OS level Node bootstrap
Chaos testing/engineering
CRI
Advanced scheduling
MetalLB
keepalived
Patching control-plane components
KEPs
Directly interacting with etcd

Skąd Kubernetes

Rok	Wydarzenie
2003–2004	**Borg** – wewnętrzny system Google
2013	Omega – następca Borga (badania)
2014	Kubernetes jako otwarta „wersja Borga”
2015	Kubernetes 1.0, powstaje **CNCF**
2016–2017	Helm, Minikube; Kubernetes wygrywa ze Swarmem i Mesosem
2022	1.24: koniec `dockershim` – kubelet mówi do containerd (CRI)

κυβερνήτης (kubernḗtēs) – sternik, pilot
k8s – 8 liter między „k” i „s”

Docker uruchamia kontenery na jednej maszynie.
Kubernetes zarządza nimi na wielu maszynach: rozmieszcza, skaluje, restartuje, łączy siecią.

Co daje Kubernetes

Stan pożądany zamiast poleceń

„mają działać 3 repliki” – Kubernetes pilnuje tego bez końca

Self-healing

restart kontenerów, nowe Pody za padnięte, odłączanie chorych węzłów

Service discovery i load balancing

stała nazwa DNS i adres przed zmieniającymi się Podami

Rolling update i rollback

aktualizacja bez przestoju, powrót jednym poleceniem

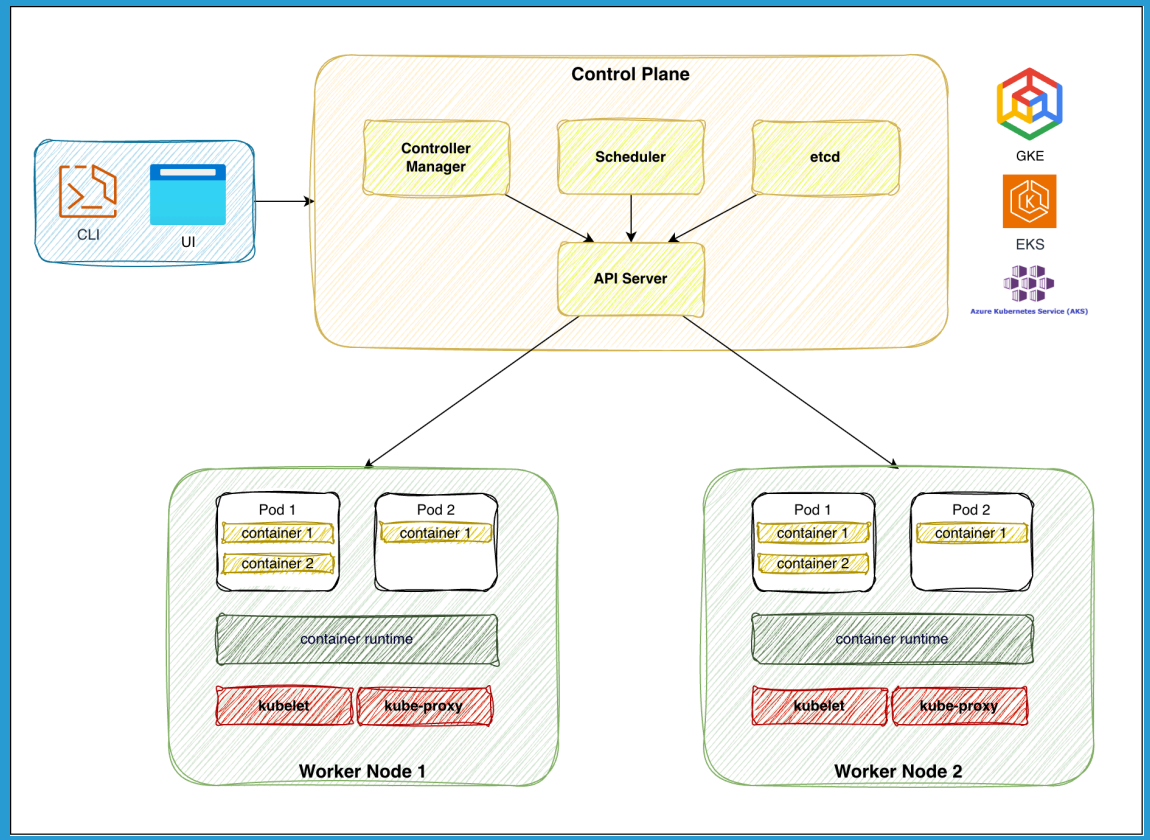
Storage i konfiguracja

woluminy na żądanie, ConfigMap, Secret

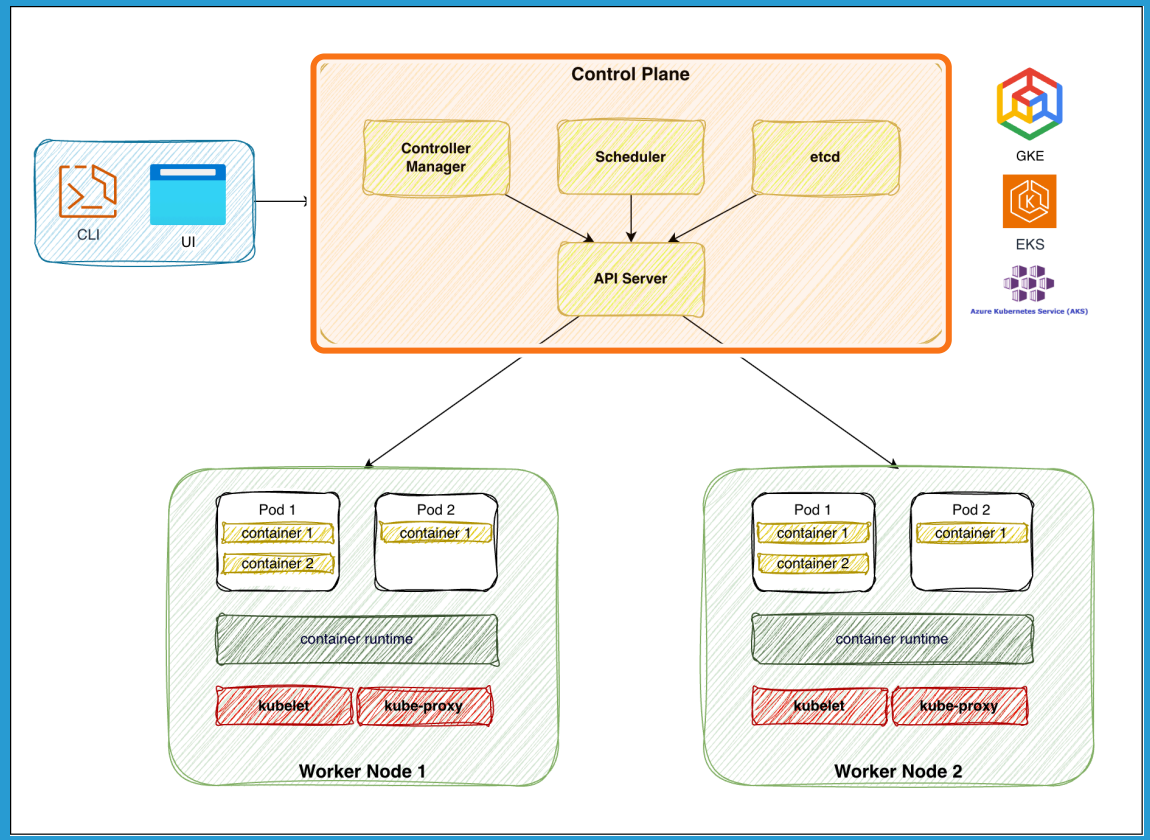
Bin packing i skalowanie

rozmieszczanie wg zasobów, HPA dokłada repliki

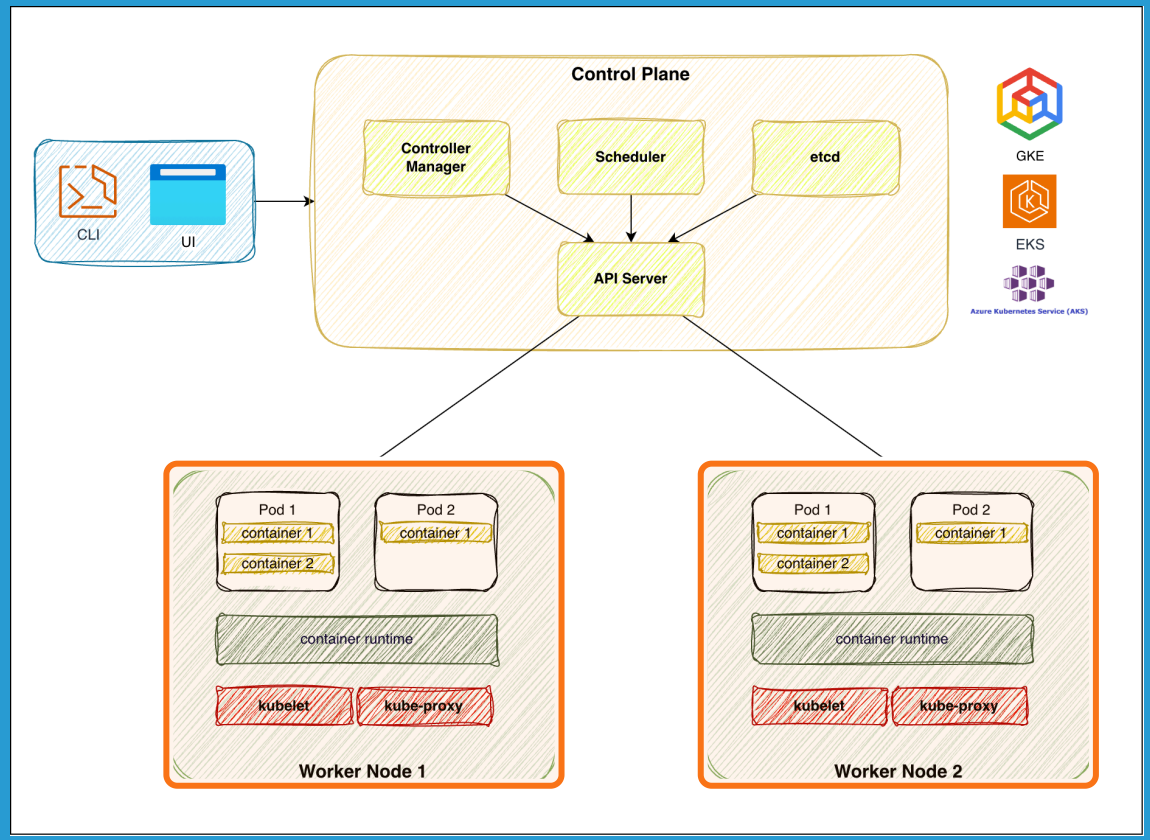
Architektura Kubernetesa



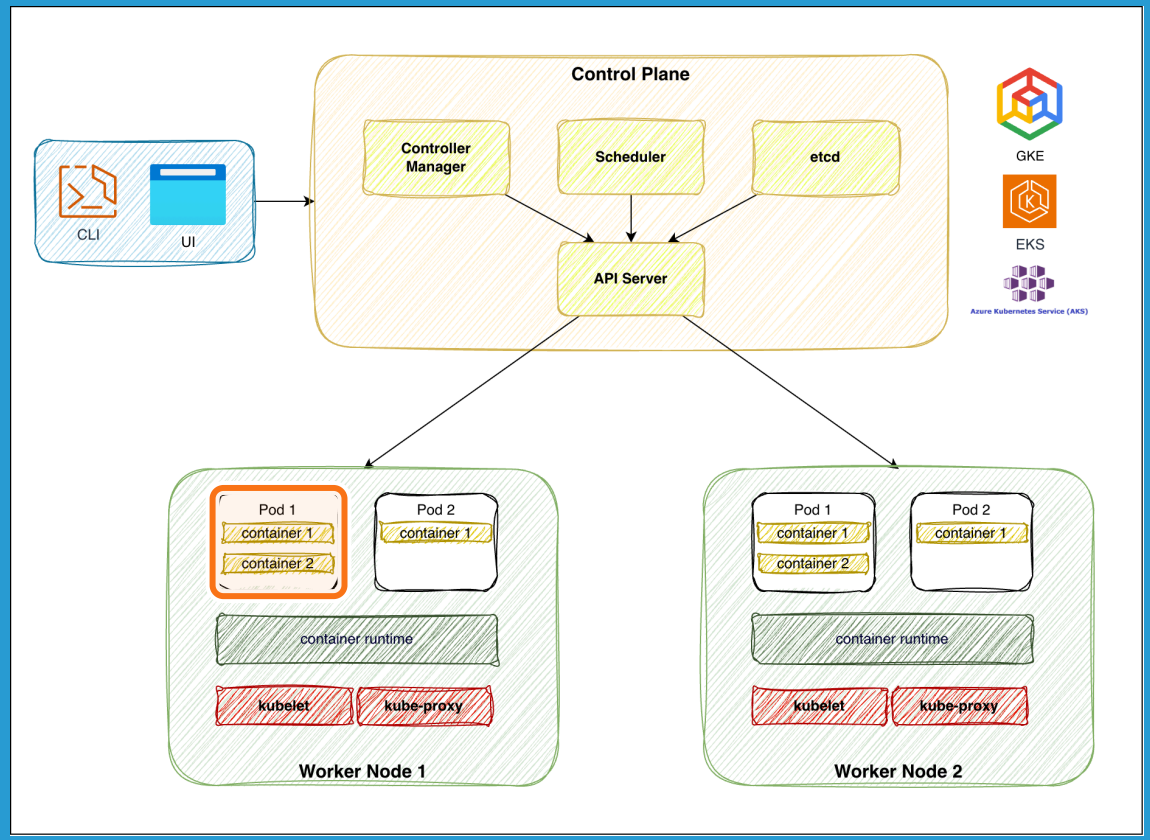
Control Plane – zarządzanie klastrem



Worker nodes – miejsce dla aplikacji



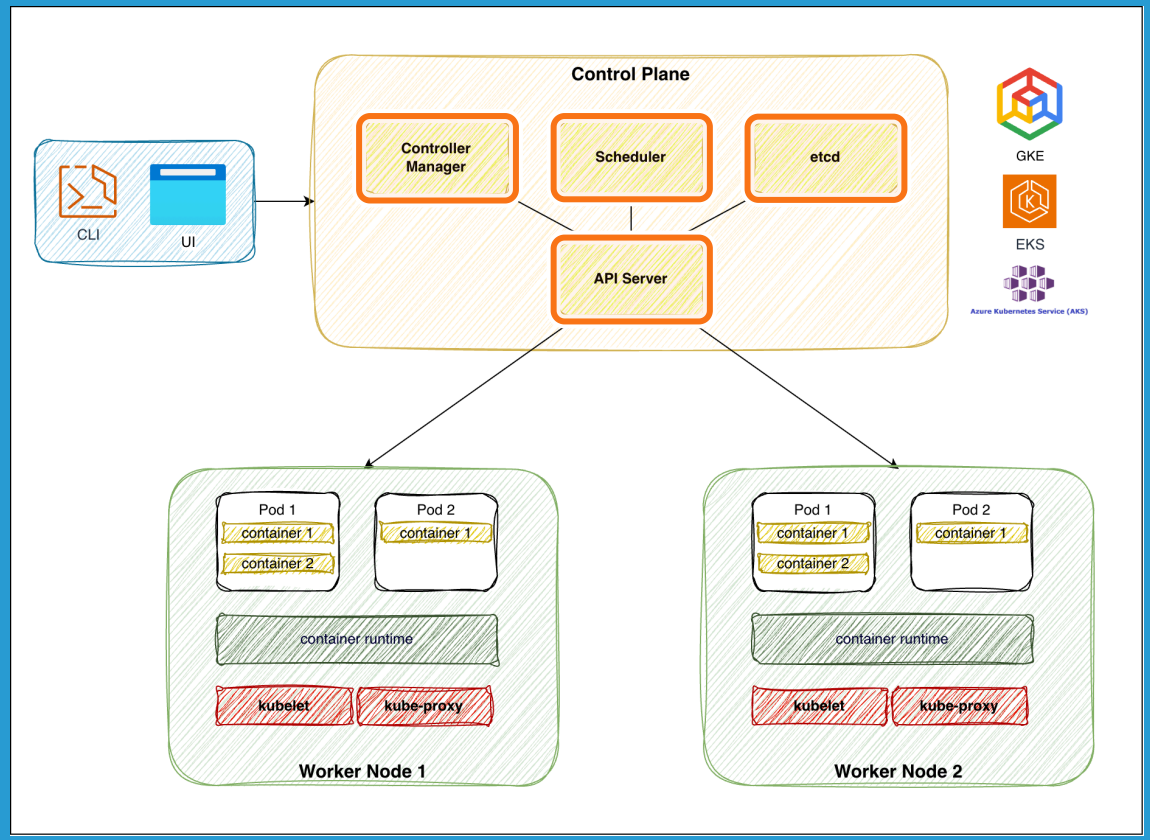
Pod – jednostka uruchamiania aplikacji



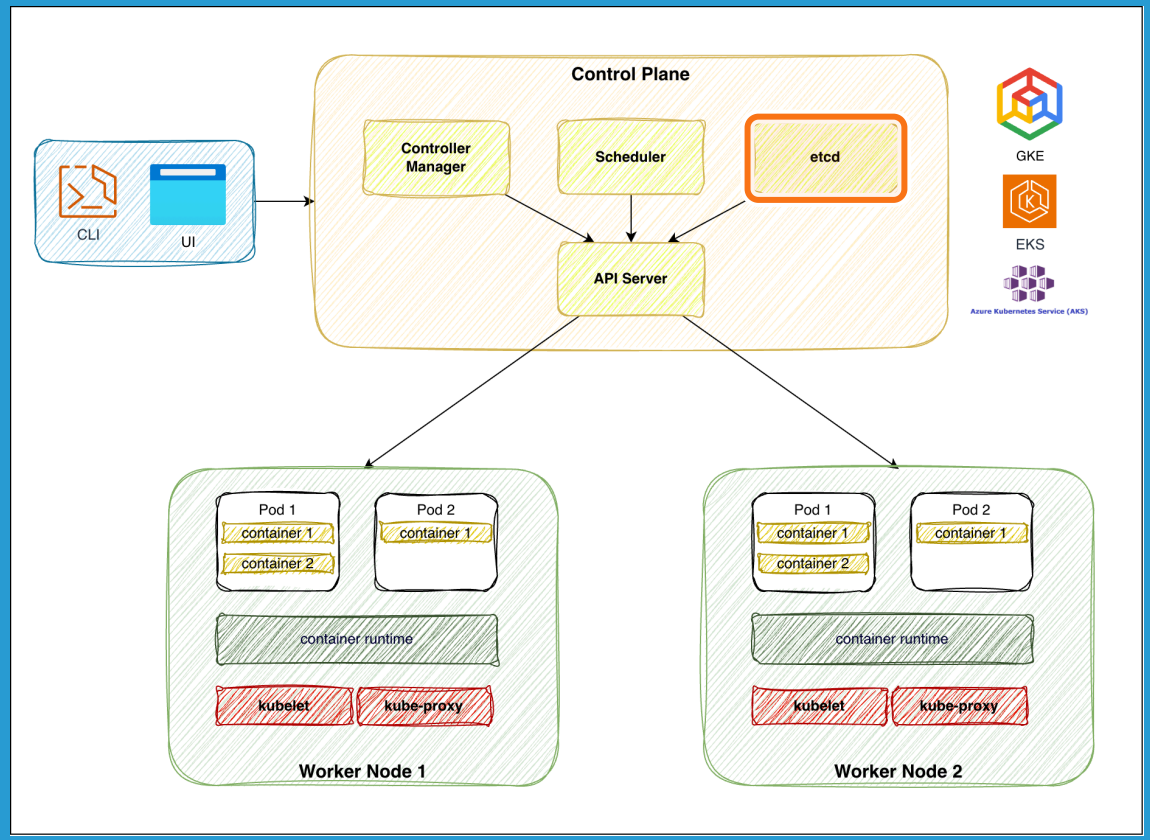
Warstwa sterowania i jej składniki

etcd · kube-apiserver · kube-controller-manager · kube-scheduler

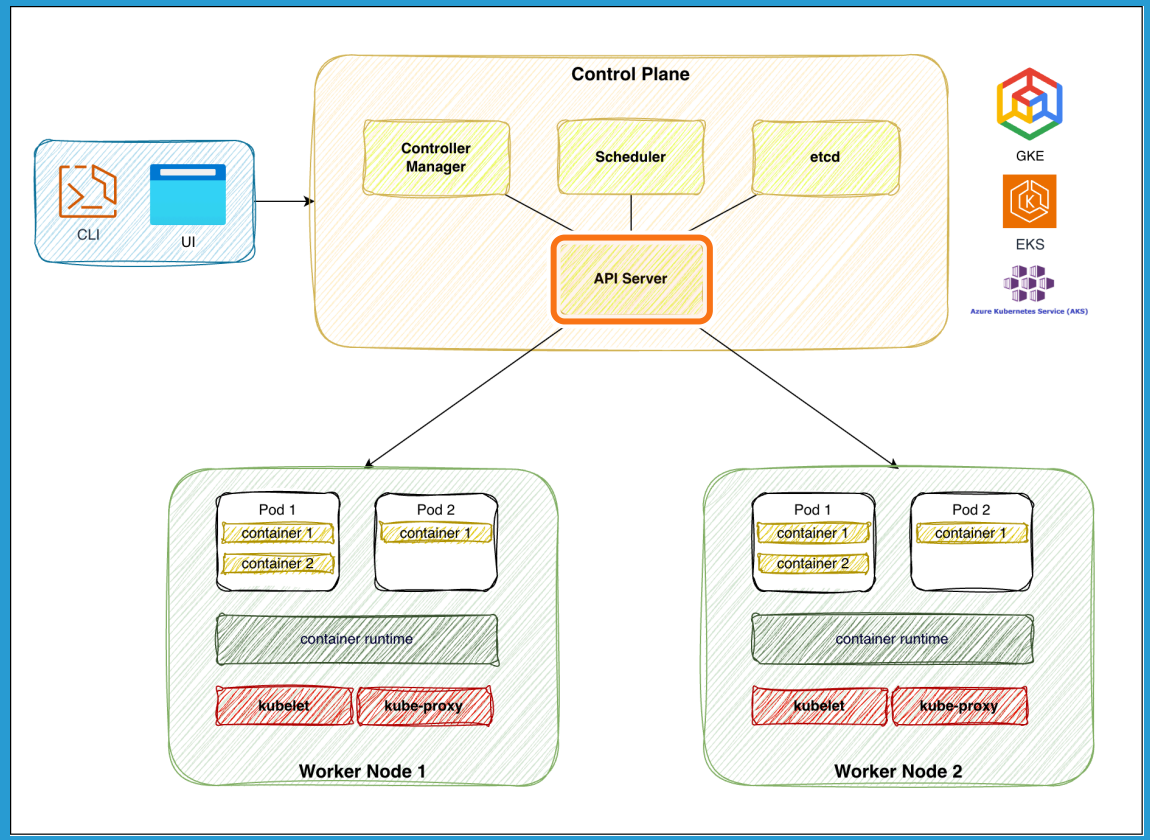
Komponenty Control Plane



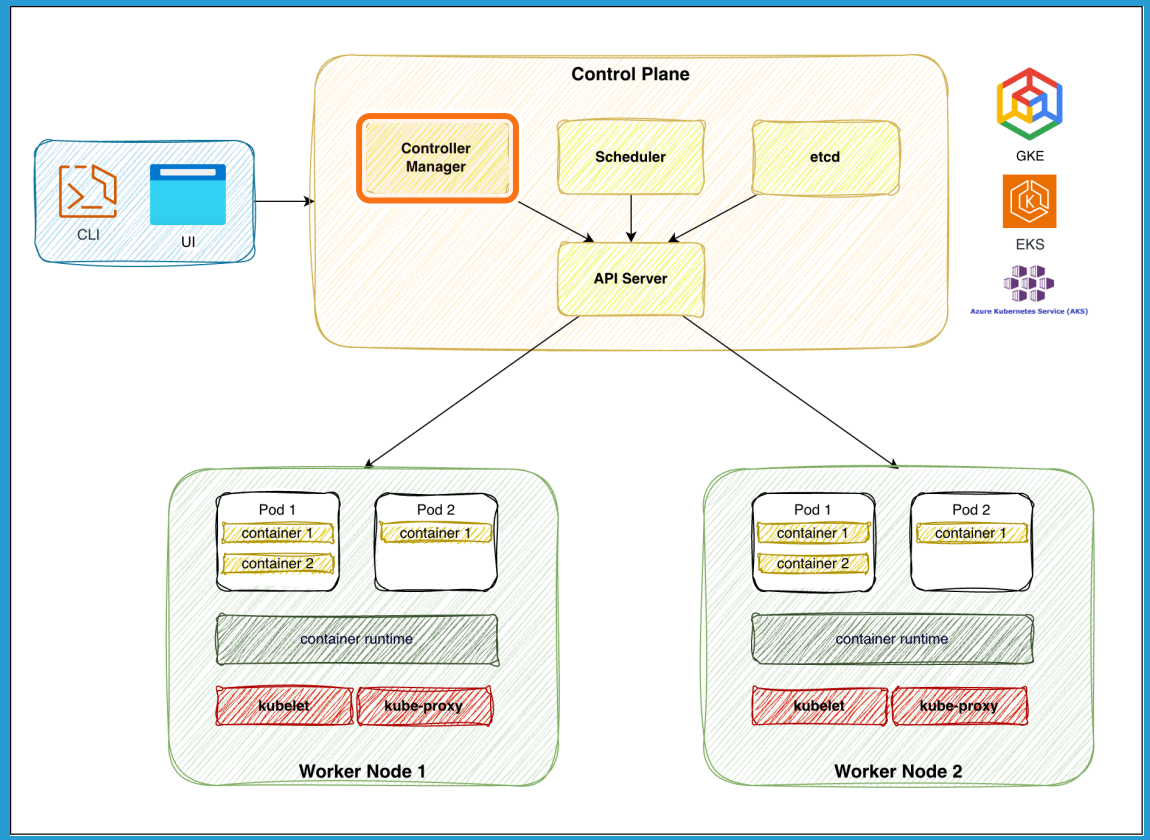
etcd – przechowywanie stanu



kube-apiserver – interfejs klastra



kube-controller-manager – kontrolery



Mapa kontrolerów Kubernetesa

kube-controller-manager

Wbudowane kontrolery — główne przykłady

Aplikacje i aktualizacje

ReplicationController
Liczba replik — starszy mechanizm

ReplicaSet
Utrzymanie liczby Podów

Deployment
ReplicaSety i rollout aplikacji

StatefulSet
Tożsamość Podów i ich PVC

DaemonSet
Pod na każdym wybranym węźle

Zadania i skalowanie

Job
Wykonanie zadania do końca

CronJob
Joby według harmonogramu

HPA
Zmiana liczby replik wg metryk

Węzły, sieć i storage

Node lifecycle
Stan węzłów i tainty

EndpointSlice
Lista endpointów dla Service

PersistentVolume
Powiązania PV ↔ PVC

Porządek i tożsamość

Namespace
Usuwanie namespace i zasobów

Garbage collector
Usuwanie obiektów zależnych

ServiceAccount
Konto default w namespace

Poza kube-controller-manager

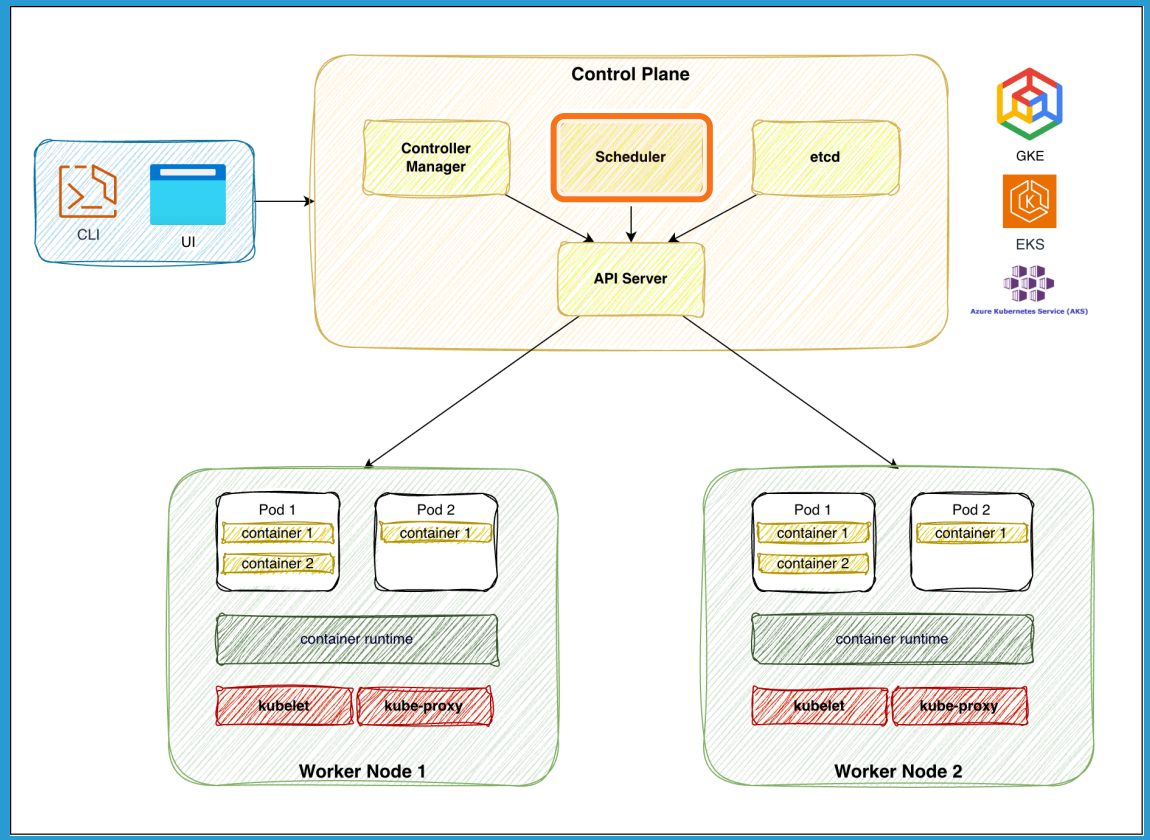
Ingress controller
Reguły Ingress → konfiguracja ruchu HTTP/HTTPS

Operator
Logika aplikacji, często oparta o CRD

Współpraca: HPA → liczba replik Deploymentu → ReplicaSet → Pody

Harmonogram: CronJob → Job → Pody

kube-scheduler – wybór węzła



Jak scheduler wybiera węzeł dla Poda?

1. Pod czeka na przydział
2. Filtrowanie: wymagania
3. Ocena: preferencje
4. Przypisanie: node-4

Wymagania Poda
requests: 2 CPU · 4 GiB RAM
nodeSelector: team=A
Preferencja: size=large (node affinity)

kube-scheduler
zapis przez API Server

node-1 · mały
team=A
size=small
1 CPU · 2 GiB
× Za mało zasobów

node-2 · duży
team=B
size=large
8 CPU · 16 GiB
× Inny team

node-3 · średni
team=A
size=medium
4 CPU · 8 GiB
Niższa ocena

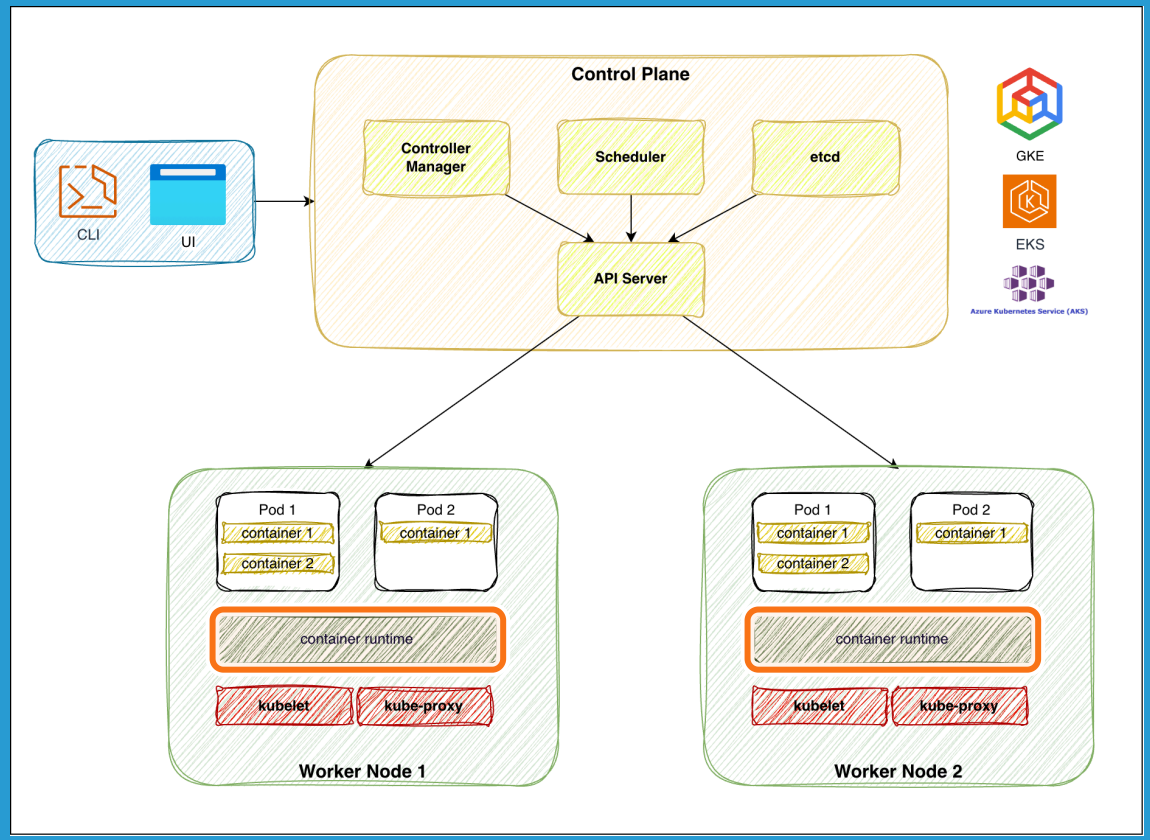
node-4 · duży
team=A
size=large
8 CPU · 16 GiB
✓ Pod przypisany
Pod

Scheduler przypisuje Pod. Dopiero kubelet i runtime uruchamiają jego kontenery.

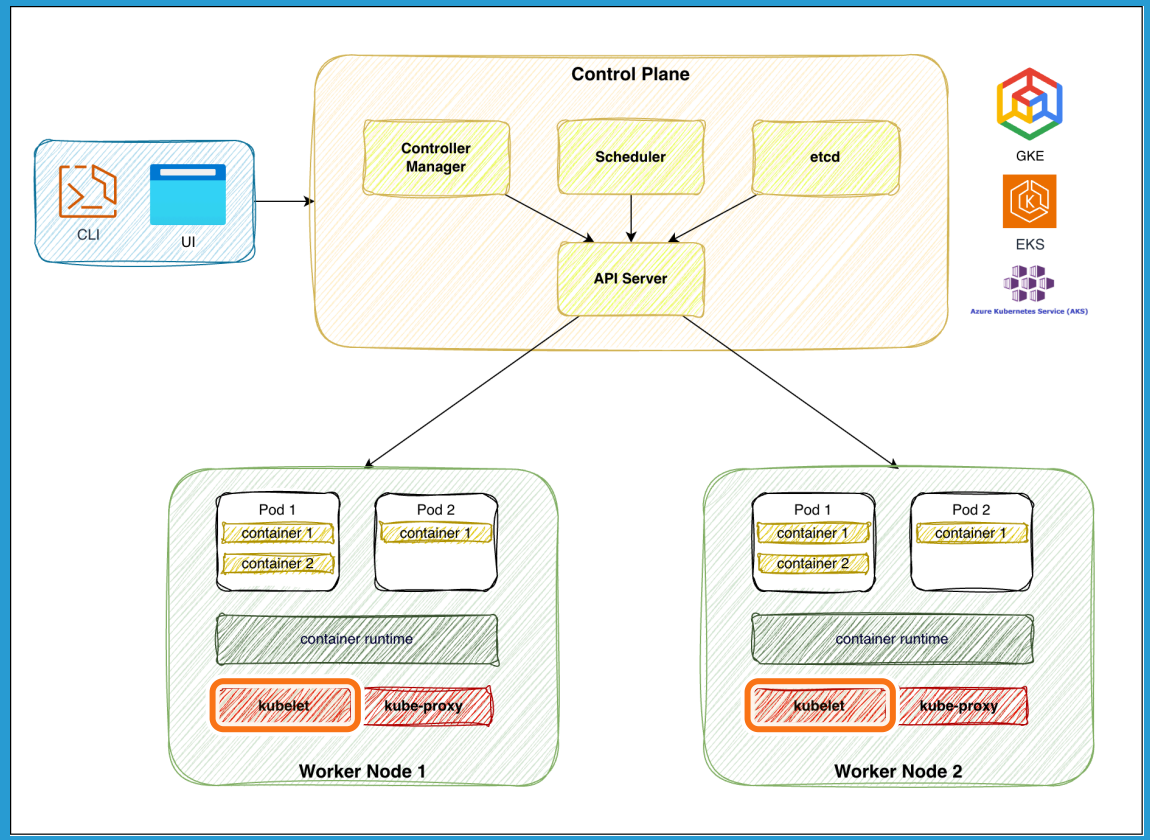
Węzły i ich składniki

container runtime · kubelet · kube-proxy

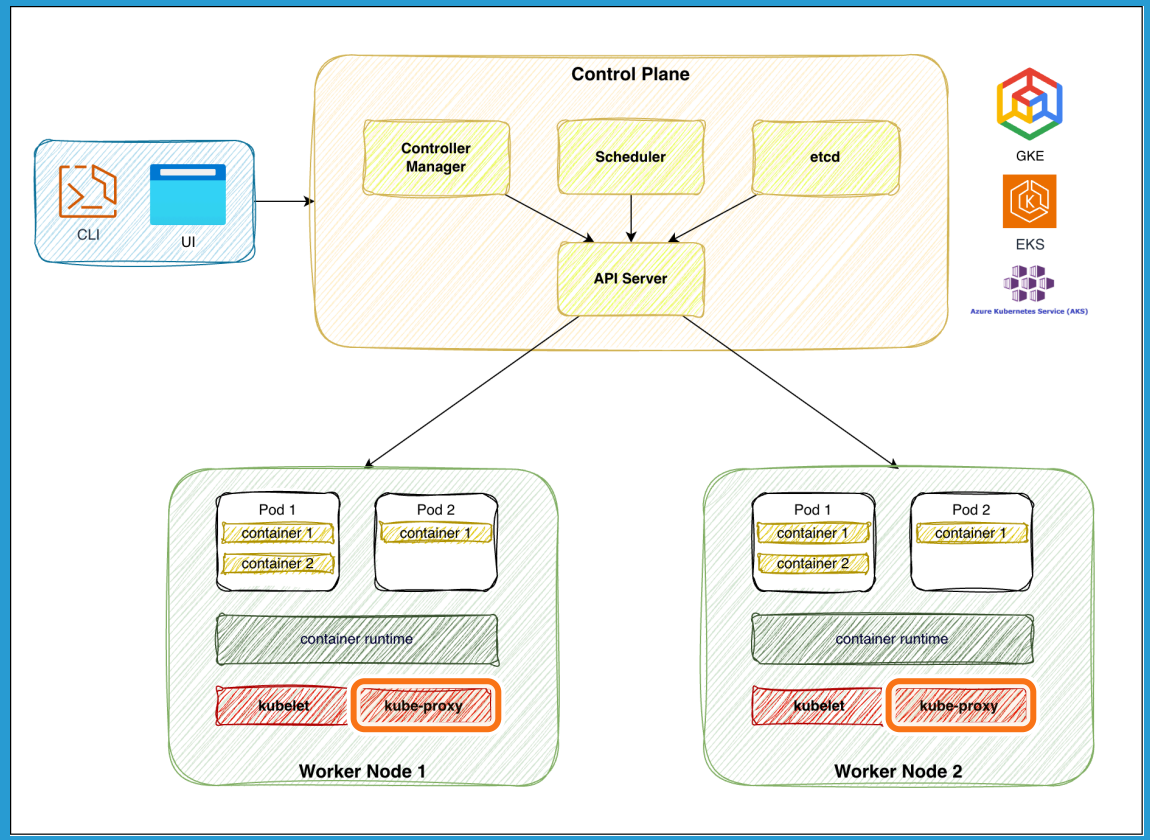
Container runtime – uruchamianie kontenerów



kubelet – opieka nad Podami



kube-proxy – ruch do usług



Od aplikacji do bazy – rola kube-proxy

3. Przekazanie do bazy

API Server
Service + EndpointSlice

← obserwuje

kube-proxy
na węźle app

→ konfiguruje

Reguły w jądrze
np. iptables / nftables

Adres docelowy (DNAT)

10.96.0.20:5432 → 10.244.2.8:5432

Węzeł A

Pod app

10.244.1.5

klient bazy danych

→ połączenie

Stały adres wirtualny

Service db

10.96.0.20:5432

nie jest osobnym Podem

→ reguły + sieć Podów

Węzeł B

Pod db

10.244.2.8:5432

nasłuchująca baza danych

Kube-proxy ustawia reguły. Pakiety przekazuje jądro systemu.

Jądro zmienia adres docelowy na 10.244.2.8:5432. Sieć Podów dostarcza ruch do bazy.

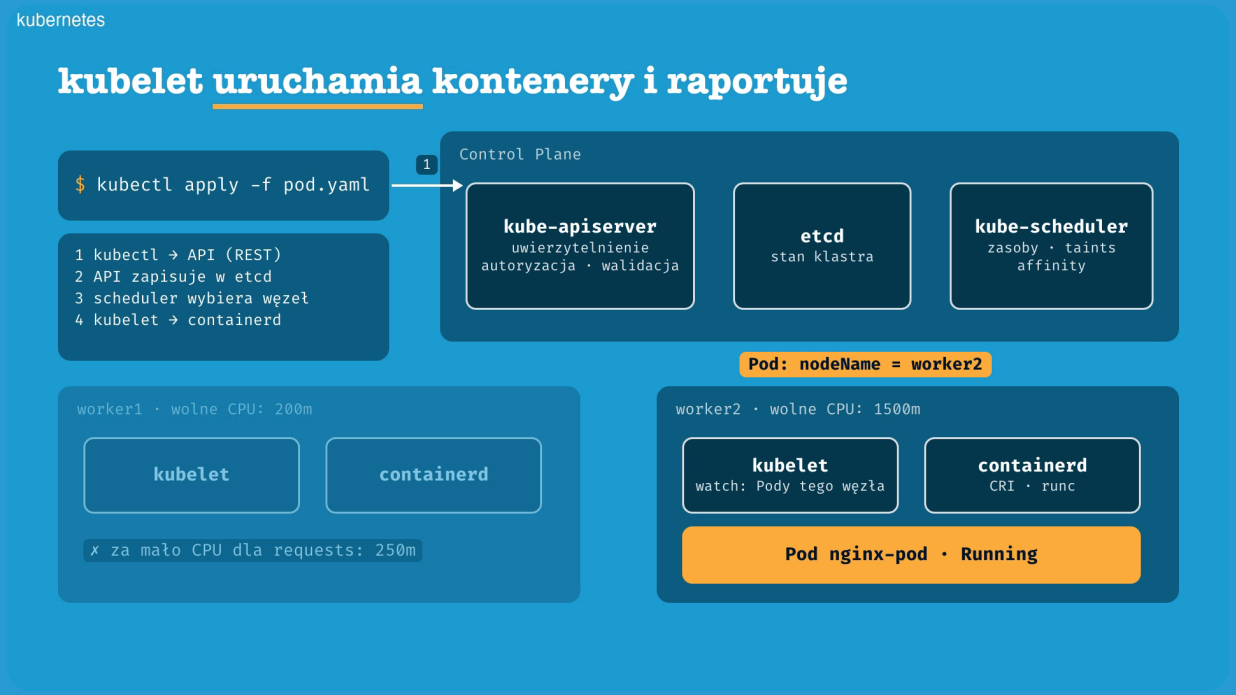
Control Plane – kubelet i container runtime

Przykład: klaster z kubeadm (i minikube)

- **kubelet i container runtime działają także na węzłach Control Plane**
- **komponenty Control Plane to static Pody z** `/etc/kubernetes/manifests`
- **kubelet i runtime uruchamiają je i utrzymują na węźle**
- **kube-proxy może działać i tutaj – zależy od konfiguracji sieci klastra**

```
minikube ssh -- sudo ls /etc/kubernetes/manifests
minikube ssh -- sudo crictl ps --name kube-
```

Przepływ tworzenia Poda



animacja: assets/animations/k8s-scheduling.mp4

Pod – sidecar, init container, cykl życia

- **sidecar – pomocniczy kontener w Podzie (proxy, agent logów)**
- **init container – uruchamia się przed głównymi (czekanie na bazę – sekcja 5)**
- **cykl życia:** `Pending` → `Running` → `Succeeded` / `Failed` (`Unknown` – brak kontaktu z węzłem)
- **Pody są efemeryczne: nowy Pod = nowa nazwa i nowy IP → stałe adresy daje Service**

Demo – klaster od środka

```
kubectl get nodes -o wide
kubectl get pods -n kube-system -o wide       # etcd, apiserver, scheduler, controller-manager…
minikube ssh -- ls /etc/kubernetes/manifests  # statyczne Pody Control Plane
kubectl create deployment hello --image=nginx:1.29 --replicas=2
kubectl delete pod -l app=hello --wait=false && kubectl get pods -l app=hello
kubectl api-resources | head · kubectl explain pod.spec.containers
```

Który komponent działa na każdym węźle? Kto tworzy nowe Pody po usunięciu?

2. Instalacja Kubernetesa

metody • minidystrybucje • minikube • kind • k3s

Metody instalacji

Dystrybucja	Opis i zastosowanie
minikube	nauka; 1 węzeł (lub `--nodes`), sterowniki Docker/VM, addony – **nasz klaster**
kind	węzły jako kontenery Dockera; CI/CD, testy, bardzo szybki start
k3s	lekki (~70 MB), produkcyjny – edge, IoT, Raspberry Pi
k0s	jedna binarka bez zależności, on-prem
MicroK8s	snap dla Ubuntu, addony
kubeadm	pełny klaster produkcyjny, pełna kontrola – **sekcja 7**
EKS, GKE, AKS	zarządzany Control Plane w chmurze

Porównanie minidystrybucji z pełnym Kubernetesem

Cecha	minikube	kind	k3s	kubeadm
Cel	nauka, dev	CI/CD, testy	edge, prod lekki	produkcja
Węzły	1 lub wiele (`--nodes`)	wiele (kontenery)	wiele	wiele (VM/bare metal)
Zasoby	~4 GB RAM	~2 GB RAM	~512 MB RAM	2+ GB/węzeł
Czas startu	~2 min	~1 min	~30 s	15–30 min
Addony	wbudowane	ręczna inst.	wbudowane	ręczna inst.
Produkcja	nie	nie	tak	tak

Ćwiczenie 2.3 – minikube na jump-boksie

```
minikube start --driver=docker --cni=calico --cpus=2 --memory=4096   # 16 GB na jump-boksie: 4 / 8192
minikube status && kubectl get nodes
minikube addons enable metrics-server
minikube addons enable ingress
minikube addons enable headlamp        # zamiast wycofanego Dashboard
kubectl -n headlamp create token headlamp --duration 24h
kubectl -n headlamp port-forward svc/headlamp 8080:80   # laptop: ssh -L 8080:localhost:8080 …
```

Wynik: węzeł `minikube` `Ready`, Headlamp pokazuje Pody `kube-system`.

kind i k3s

kind – węzły to kontenery

```
kind create cluster --name moj-klaster \
  --config kind-config.yaml
kubectl get nodes
docker ps --filter \
  label=io.x-k8s.kind.cluster=moj-klaster
```

k3s – jedno polecenie (osobna maszyna Linux)

```
curl -sfL https://get.k3s.io | sh -
sudo k3s kubectl get nodes
```

Ćwiczenie 2.5 – klaster kind z 3 węzłami

```
kind create cluster --name lab-klaster --config klaster.yaml   # 1 CP (80/443) + 2 workery
kubectl get nodes -o wide
kubectl get pods -n kube-system
docker build -t moja-app:latest ../../app-node
kind load docker-image moja-app:latest --name lab-klaster
kind delete cluster --name lab-klaster && kubectl config use-context minikube
```

Wynik: 3 węzły `Ready`, obraz w containerd węzłów (`crictl images`).

Kilka klastrów: konteksty, kubectx, kubens

kubectl – konteksty z `~/.kube/config`

```
kubectl config get-contexts
kubectl config use-context kind-lab-klaster
kubectl --context minikube get nodes
kubectl config set-context --current \
  --namespace=kube-system
```

kubectx / kubens – to samo krócej

```
kubectx                     # lista
kubectx kind-lab-klaster
kubectx -                   # poprzedni
kubens kube-system
kubens -
```

Instalacja: `konteksty.sh` • `apt install kubectx` • `krew install ctx ns`

Autouzupełnianie i kontekst w prompcie ★

bash

```
cat >> ~/.bashrc <<'EOF'
source <(kubectl completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k
EOF
```

zsh + oh-my-zsh (z części Docker)

```
# ~/.zshrc
plugins=(git docker kubectl minikube kind)
```

starship – moduł `kubernetes`

```
# ~/.config/starship.toml
[kubernetes]
disabled = false        # domyślnie wyłączony

[[kubernetes.contexts]]
context_pattern = ".*prod.*"
style = "bold red"
```

```
~ ☸ minikube (notes) ❯
```

3. Podstawowe operacje

Pod · Deployment · skalowanie · HPA · rolling update · rollback · canary

kubectl delete pod notes-api-...



Deployment: I'll be back.

Imperatywnie czy deklaratywnie

Imperatywnie – szybkie testy

```
kubectl run test-pod --image=nginx:1.29
kubectl create deployment web \
  --image=nginx:1.29 --replicas=2
kubectl delete pod test-pod
```

Deklaratywnie – produkcja, Git

```
kubectl apply -f deployment.yaml
kubectl diff -f deployment.yaml
kubectl delete -f deployment.yaml
```

Most między nimi: `kubectl create deployment web --image=nginx:1.29 --dry-run=client -o yaml > web.yaml`

Zasoby Kubernetesa – ściąga

Zasób	Skrót	Opis
Pod	po	1+ kontenerów, wspólna sieć i storage
Deployment	deploy	repliki, rolling update, rollback
ReplicaSet	rs	utrzymuje liczbę replik
StatefulSet	sts	stałe nazwy i woluminy (bazy)
DaemonSet	ds	1 Pod na każdym węźle
Job · CronJob	job · cj	zadanie do skutku · wg harmonogramu
Service	svc	stały endpoint dla Podów
Ingress	ing	routing HTTP/HTTPS do Serwisów

Zasób	Skrót	Opis
ConfigMap	cm	konfiguracja (nie-wrażliwa)
Secret	secret	hasła, tokeny (base64)
Namespace	ns	wirtualny podział klastra
PersistentVolume	pv	zasób storage w klastrze
PersistentVolumeClaim	pvc	żądanie storage przez Pod
StorageClass	sc	typ storage, provisioner
HorizontalPodAutoscaler	hpa	skalowanie wg CPU/RAM/metryk

Pod z manifestu YAML

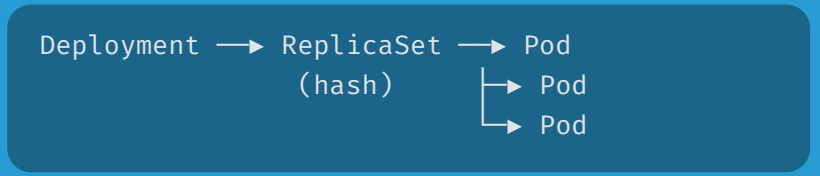
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels: { app: nginx, env: dev }
spec:
  containers:
  - name: nginx
    image: nginx:1.28
    ports:
    - containerPort: 80
    resources:
      requests: { memory: "64Mi", cpu: "250m" }
      limits:   { memory: "128Mi", cpu: "500m" }
    livenessProbe:
      httpGet: { path: /, port: 80 }
      initialDelaySeconds: 5
      periodSeconds: 10
```

```
kubectl apply -f pod.yaml
kubectl get pods -o wide
kubectl describe pod nginx-pod
kubectl logs nginx-pod
kubectl exec nginx-pod -- nginx -v
kubectl port-forward pod/nginx-pod 8080:80
```

- **requests – rezerwacja dla schedulera**
- **limits – górna granica (CPU: dławienie, RAM: OOMKill)**
- `250m` **= 0,25 CPU** · `Mi` **= mebibajty**

Deployment – zarządzanie replikami

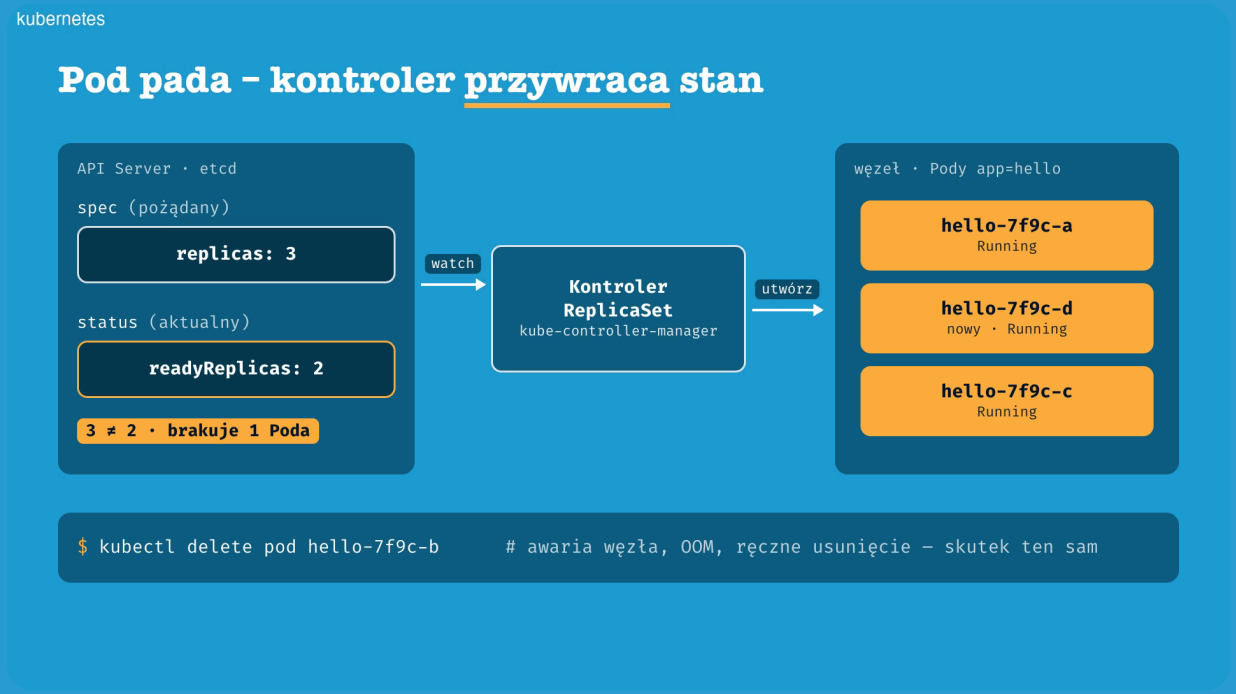
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels: { app: nginx }
  template:
    metadata:
      labels: { app: nginx }
    spec:
      containers:
      - name: nginx
        image: nginx:1.28
        ports: [{ containerPort: 80 }]
```




```
kubectl apply -f deployment.yaml
kubectl get deploy,rs,pods
kubectl get pods --show-labels
```

- `selector` **musi pasować do** `template.labels`
- **każda zmiana** `template` **= nowy ReplicaSet**

Stan pożądany i pętla kontrolera



animacja: assets/animations/k8s-reconcile.mp4

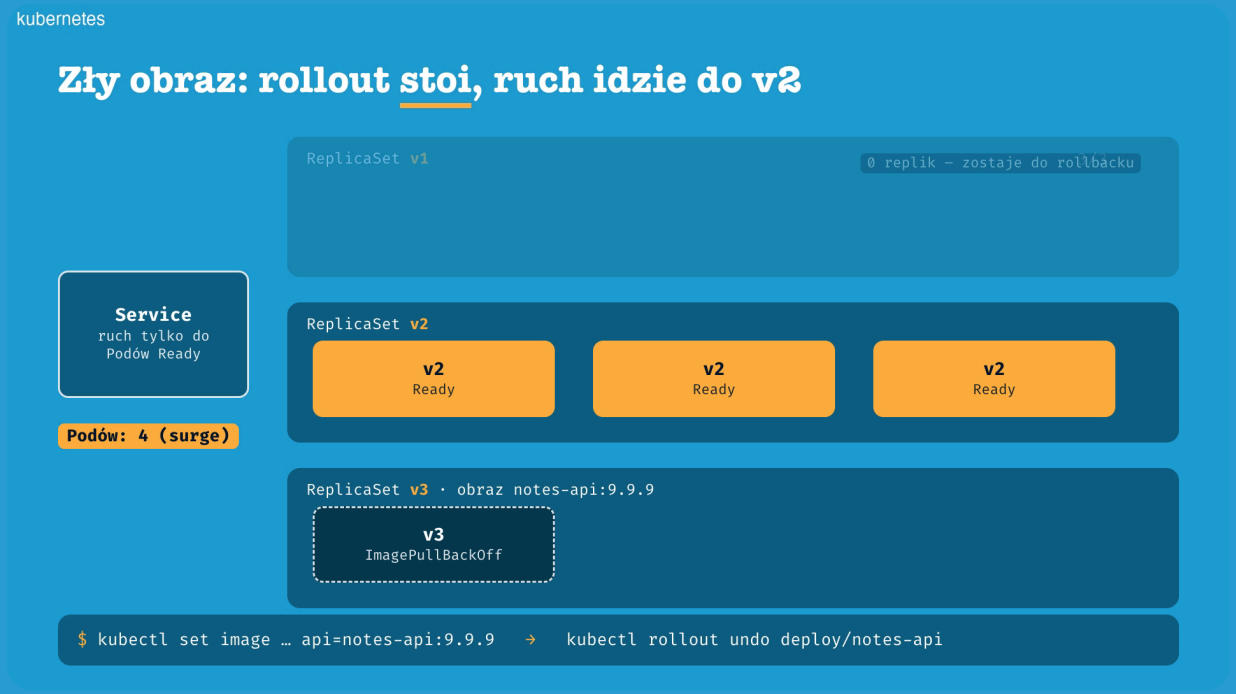
Skalowanie: ręcznie i HPA

```
kubectl scale deploy nginx-deployment \
  --replicas=5
kubectl get pods -w

# addon metrics-server
kubectl top pods
kubectl autoscale deploy nginx-deployment \
  --cpu=50% --min=2 --max=10
kubectl get hpa
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: nginx-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Rolling update i rollback



animacja: assets/animations/k8s-rolling-update.mp4

Aktualizacje – polecenia

```
D=deploy/nginx-deployment
kubectl set image $D nginx=nginx:1.29
kubectl annotate $D --overwrite \
  kubernetes.io/change-cause="nginx 1.29"
kubectl rollout status $D
kubectl rollout history $D
kubectl rollout history $D --revision=2
kubectl rollout undo $D
kubectl rollout undo $D --to-revision=2
```

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1        # ponad replicas
      maxUnavailable: 0  # niedostępne w trakcie
```

- `Recreate` – **najpierw wszystkie stare w dół (przestój)**
- **każdy rollback = nowa rewizja**

Canary deployment

```
Service app=nginx-canary-demo
  ├── nginx-stable  (track=stable)  9 × nginx:1.28
  └── nginx-canary  (track=canary)  1 × nginx:1.29
```

```
kubectl apply -f canary/
kubectl get pods -l app=nginx-canary-demo -L track
kubectl run licznik --image=busybox:1.37 --rm -i --restart=Never -- \
  sh -c 'for i in $(seq 50); do wget -S -q -O /dev/null \
  http://nginx-canary-demo 2>&1 | grep Server:; done | sort | uniq -c'
```

Wynik: ok. 45 × `nginx/1.28.x`**, ok. 5 ×** `nginx/1.29.x`

podział ruchu = proporcja replik

precyzyjne wagi (1%, nagłówki): Gateway API, Argo Rollouts, service mesh

Namespaces, etykiety, adnotacje ★

```
kubectl create namespace dev
kubectl get pods -n dev
kubectl get pods -A
kubectl config set-context --current \
  --namespace=dev
kubectl delete namespace dev
```

```
kubectl get pods --show-labels
kubectl get pods -l app=nginx,tier=frontend
kubectl get pods -l 'env in (dev,test)'
kubectl label pod X app=debug --overwrite
```

- **namespace – podział na zespoły/środowiska, zakres RBAC, quot i NetworkPolicy; nie izolacja sieci**
- **etykiety – do selekcji: Service, Deployment, NetworkPolicy wybierają Pody etykietami**
- **adnotacje – metadane dla ludzi i narzędzi (`change-cause`, konfiguracja Ingressu)**
- **Pod ze zmienioną etykietą wypada z ReplicaSetu – a RS tworzy brakującą replikę**

ConfigMap i Secret ★

```yaml
envFrom:
- configMapRef: { name: app-config }  # klucze → zmienne
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef: { name: app-secret, key: DB_PASSWORD }
volumeMounts:
- { name: config, mountPath: /config } # klucze → pliki
volumes:
- name: config
  configMap: { name: app-config }
```

```
kubectl create configmap demo \
  --from-literal=KOLOR=zielony
kubectl create secret generic demo \
  --from-literal=HASLO=tajne
kubectl get secret demo -o \
  jsonpath='{.data.HASLO}' | base64 -d
```

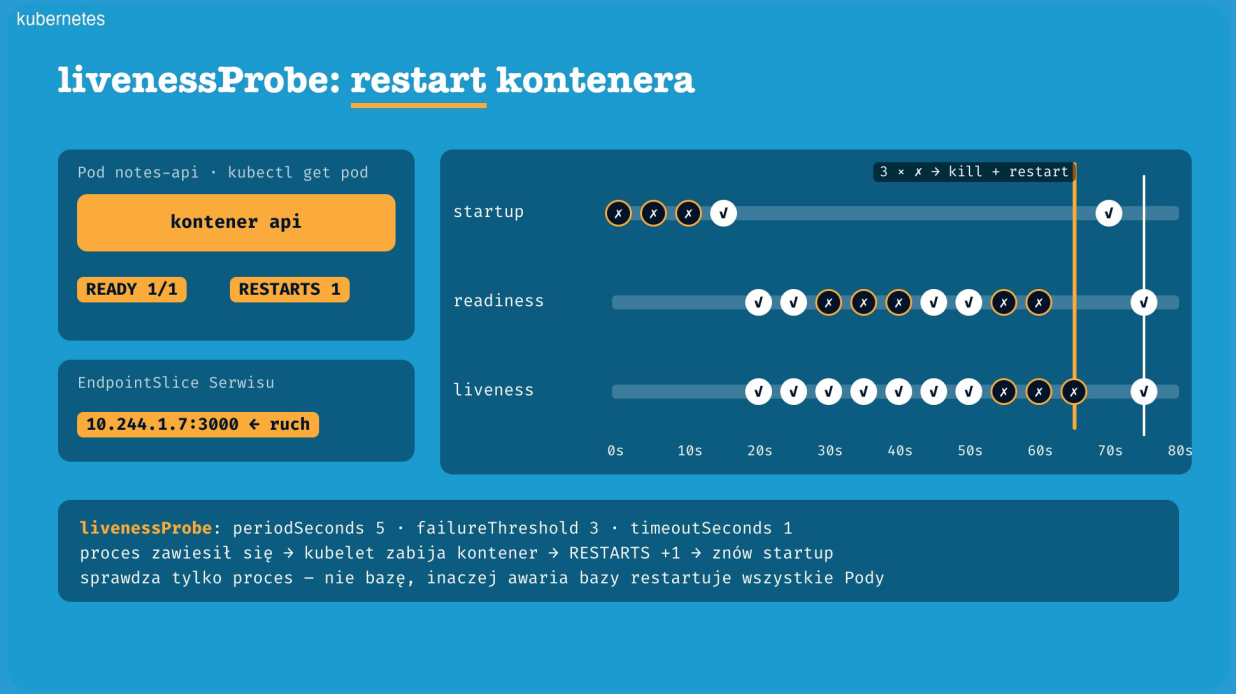
- **Secret to base64, nie szyfrowanie – chroni go RBAC i szyfrowanie etcd**
- **zmiana ConfigMap: pliki odświeżają się same (~1 min), zmienne dopiero po restarcie Poda**

Sondy: readiness, liveness, startup ★

Sonda	Gdy nie przechodzi	notes-api
readinessProbe	Pod znika z Serwisu — nie dostaje ruchu	`GET /readyz` (baza odpowiada)
livenessProbe	kubelet restartuje kontener	`GET /healthz` (proces żyje)
startupProbe	wstrzymuje pozostałe sondy do pierwszego sukcesu	wolny start (Java)

```
kubectl apply -f sondy.yaml
kubectl get pods -l app=sondy -w                # readiness: 0/1 Running · liveness: RESTARTS rośnie
kubectl exec sonda-readiness -- touch /tmp/gotowe   # po chwili 1/1
```

Sondy w akcji ★



animacja: assets/animations/k8s-probes.mp4

Rodzaje sond i wspólne parametry ★

Mechanizm	Sukces
`httpGet`	kod HTTP 200–399
`tcpSocket`	port przyjmuje połączenie
`exec`	polecenie kończy się kodem 0
`grpc`	`SERVING` (gRPC Health)

Każdą sondę wykonuje kubelet na węźle Poda. Mechanizm i parametry są takie same dla startup, readiness i liveness.

Parametr	Domyślnie	Znaczenie
`initialDelaySeconds`	0	opóźnienie 1. próby od startu
`periodSeconds`	10	co ile sekund próba
`timeoutSeconds`	1	brak odpowiedzi = porażka
`failureThreshold`	3	porażki z rzędu → sonda nie przechodzi
`successThreshold`	1	sukcesy z rzędu → przechodzi

Czas reakcji ≈ `failureThreshold × periodSeconds` – przy domyślnych 3 × 10 s = 30 s.

startupProbe – ochrona wolnego startu ★

```yaml
startupProbe:
  httpGet: { path: /healthz, port: 3000 }
  periodSeconds: 10
  failureThreshold: 30   # limit: 30 × 10 s = 5 min
livenessProbe:           # rusza po 1. sukcesie startup
  httpGet: { path: /healthz, port: 3000 }
  periodSeconds: 10
  failureThreshold: 3
```

- **działa tylko na starcie – po 1. sukcesie już nie (do restartu kontenera)**
- **do tego czasu readiness i liveness czekają**
- **limit minął → kubelet restartuje kontener**
- `successThreshold` **musi być 1**
- **zamiast dużego** `initialDelaySeconds` **w liveness**
- **kiedy: Java/Spring, migracje przy starcie, ładowanie modelu lub cache**

readinessProbe – czy Pod dostaje ruch ★

```yaml
readinessProbe:
  httpGet: { path: /readyz, port: 3000 }
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3     # ~15 s → poza Serwisem
  successThreshold: 2     # 2 × ✓ → z powrotem
```

- działa przez całe życie kontenera
- ✗: Pod `0/1`, adres znika z EndpointSlice – Service i Ingress nie kierują ruchu, bez restartu
- ✓: adres wraca – jedyna sonda, w której `successThreshold` > 1 ma sens
- rolling update czeka, aż nowy Pod będzie Ready
- bez sondy: Pod Ready od razu po starcie kontenera – ruch trafia, zanim aplikacja wstanie

livenessProbe – czy proces żyje ★

```yaml
livenessProbe:
  httpGet: { path: /healthz, port: 3000 }
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3    # ~30 s → restart
```

```
kubectl get pod -w                  # RESTARTS rośnie
kubectl describe pod X | grep -A3 Liveness
kubectl logs X --previous           # logi sprzed restartu
```

- **działa przez całe życie kontenera (po startup)**
- **✗: kubelet zabija kontener i uruchamia go ponownie – ten sam Pod, węzeł i IP**
- **częste restarty → `CrashLoopBackOff`**
- **`successThreshold` musi być 1**
- **sprawdza tylko proces – lekki endpoint, bez bazy**
- **za krótki `timeoutSeconds` pod obciążeniem → restarty w najgorszym momencie**
- **bez sondy: restart tylko, gdy proces się zakończy**

Ćwiczenie – notes-api w klastrze ★

1. `minikube image load notes-api:1.0.0` **(i tag** `1.1.0` **)**
2. **namespace** `notes` **; Deployment ze** `start/` **+ ConfigMap (** `STORAGE=json` **),** `imagePullPolicy: IfNotPresent` **, sondy** `/readyz` **i** `/healthz` **, zasoby,** `maxUnavailable: 0`
3. **test:** `kubectl -n notes port-forward deployment/notes-api 8000:3000`
4. **aktualizacja do** `1.1.0` **– w jakiej kolejności ConfigMap i obraz?**
5. **obraz** `notes-api:9.9.9` **– czy aplikacja nadal działa?** `rollout undo`

4. Networking w Kubernetesie

model sieci · CNI · Service · DNS · Ingress · NetworkPolicy · Gateway API

Model sieci i CNI

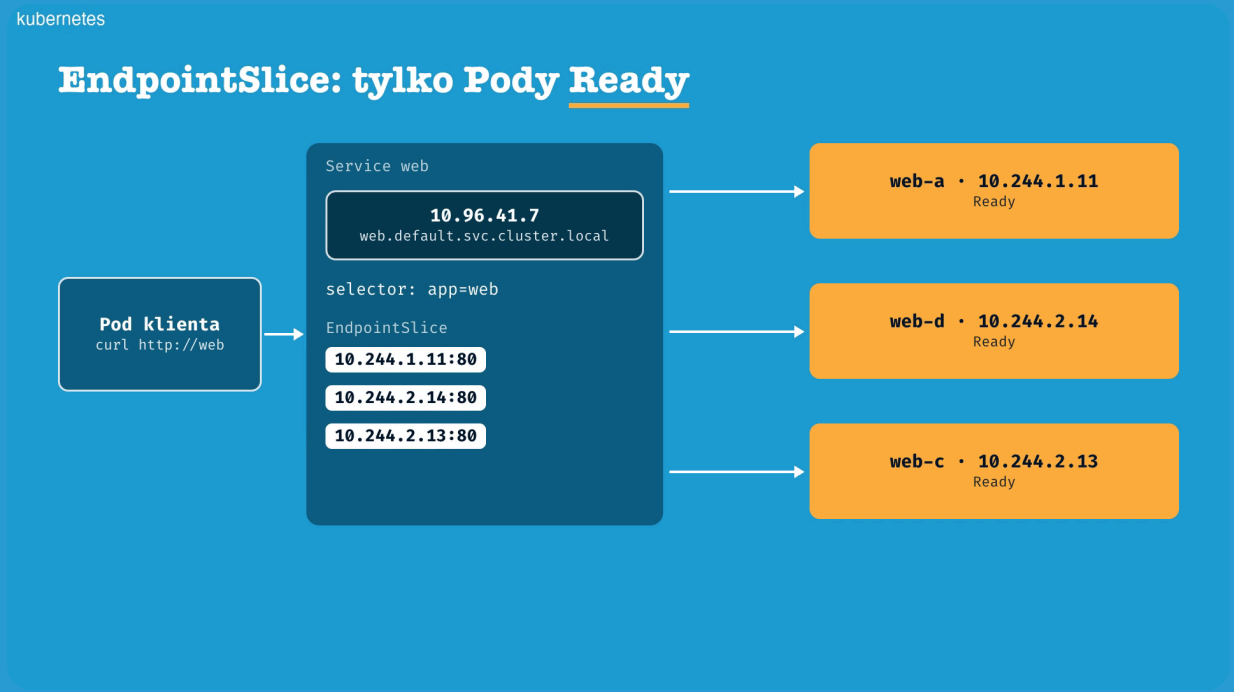
Trzy zasady sieci Kubernetesa:

1. **każdy Pod ma własny adres IP – Pody rozmawiają bez NAT**
2. **Pody na jednym węźle widzą Pody na innych węzłach – routing w warstwie CNI**
3. **agenci węzła (kubelet) widzą Pody na swoim węźle**

Sieć Podów dostarcza wtyczka CNI (Container Network Interface).

CNI	Cechy
Calico	NetworkPolicy, BGP, wydajność – **u nas**
Cilium	eBPF, polityki L7, observability (Hubble)
Flannel	prosty overlay, **bez** NetworkPolicy
Weave Net	szyfrowanie – projekt już nierozwijany

Service – stały adres przed Podami



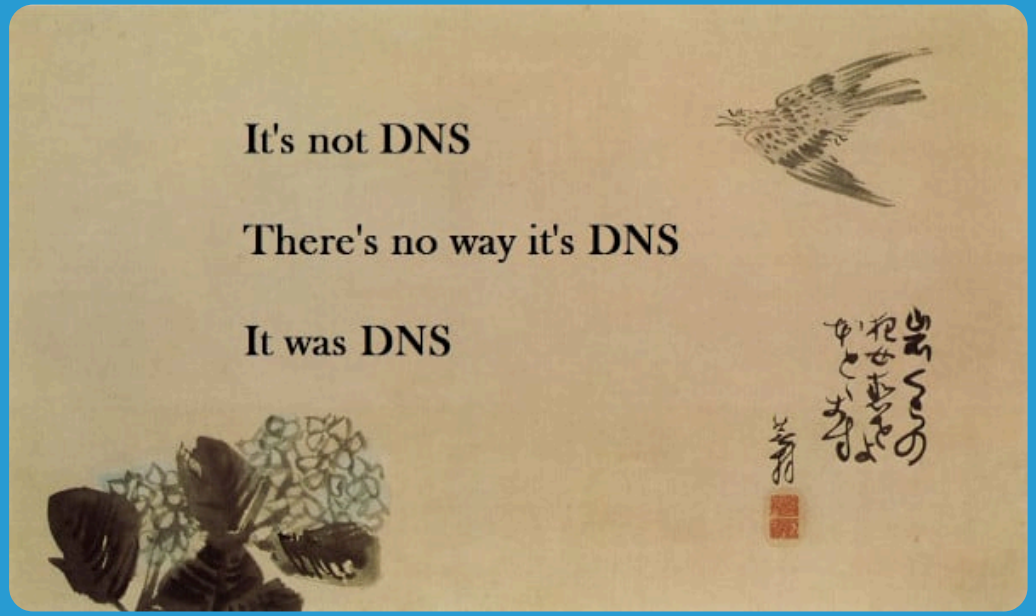
animacja: assets/animations/k8s-service.mp4

Typy Serwisów

Typ	Zastosowanie
ClusterIP	domyślny; adres tylko wewnątrz klastra
NodePort	port 30000–32767 na każdym węźle: `<NodeIP>:<port>`
LoadBalancer	zewnętrzny LB w chmurze (minikube: `minikube tunnel`)
ExternalName	alias DNS (CNAME) na nazwę spoza klastra
headless (`None`)	DNS zwraca adresy Podów – StatefulSet (sekcja 5)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx        # Pody z tą etykietą
  ports:
  - protocol: TCP
    port: 80          # port Serwisu
    targetPort: 80    # port kontenera
  type: ClusterIP
```

Coś nie działa w sieci klastra



Namespace i DNS w klastrze

Namespace – przestrzeń nazw: nazwa obiektu jest unikalna tylko w swoim namespace. Dzieli klaster na zespoły lub środowiska; nie izoluje sieci.

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: staging
```

```
kubectl apply -f namespace.yaml -f products.yaml
kubectl exec toolbox -- curl -s http://products.staging/get
```

Nazwa Serwisu `products` (ns `staging`)	Kiedy działa
`products`	z Poda w `staging`; toolbox w `default` – błąd
`products.staging`	z dowolnego namespace
`products.staging.svc.cluster.local`	pełna nazwa (FQDN)
`notes-db-0.notes-db.notes.svc.cluster.local`	Pod StatefulSetu (headless, sek. 5)

Ingress – routing HTTP do Serwisów

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: moja-aplikacja-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: frontend-svc, port: { number: 80 } }
      - path: /api
        pathType: Prefix
        backend:
          service: { name: backend-svc, port: { number: 80 } }
```

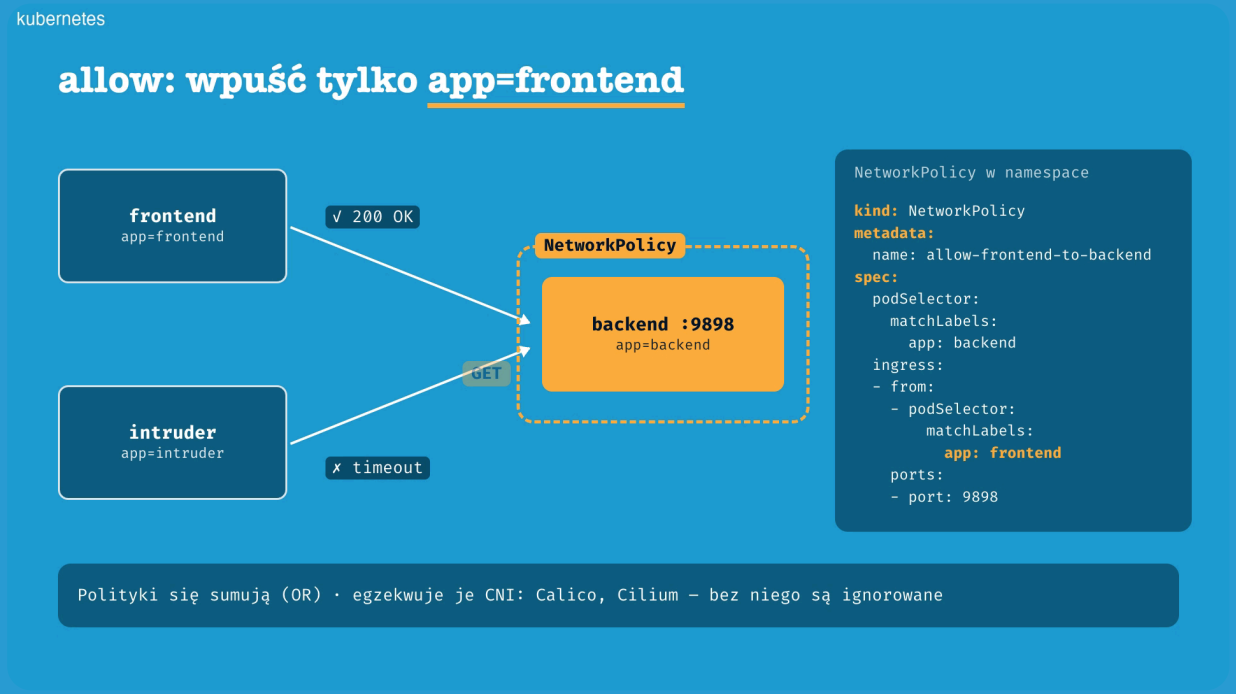
- **reguły: host + ścieżka → Service**
- **wymaga kontrolera (Ingress NGINX, Traefik, HAProxy) – minikube:** `addons enable ingress`
- **TLS: sekcja** `tls` **+ Secret z certyfikatem**
- **ingress-nginx zakończył rozwój (marzec 2026) – w nowych projektach Gateway API ★**

NetworkPolicy – firewall dla Podów

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: zezwol-tylko-frontend
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend }
    ports:
    - { protocol: TCP, port: 80 }
```

- **bez polityk: wszystko wolno**
- **Pod objęty polityką: wolno tylko to, co wymienione**
- `from`: `podSelector`, `namespaceSelector`, `ipBlock`
- `port` **= port kontenera, nie Serwisu**
- **egzekwuje CNI – Flannel/Kindnet ignorują polityki bez błędu**

NetworkPolicy w akcji



animacja: assets/animations/k8s-netpol.mp4

Ćwiczenie 4.3 – konfiguracja sieci

```
kubectl create deployment frontend --image=nginx:1.29 --replicas=2
kubectl create deployment backend  --image=httpd:2.4  --replicas=2
kubectl expose deployment frontend --port=80 --name=frontend-svc
kubectl expose deployment backend  --port=80 --name=backend-svc
kubectl run debug --image=busybox:1.37 --rm -it -- sh   # wget, nslookup
kubectl apply -f ingress.yaml
curl -H "Host: app.example.com" http://$(minikube ip)/api
kubectl apply -f network-policy.yaml
kubectl run unauthorized --image=busybox:1.37 --rm -i \
  --restart=Never -- wget -qO- -T 5 http://backend-svc
```

Wynik: Ingress `/` → nginx, `/api` → „It works!”; po polityce `unauthorized` – timeout, a `/api` przez Ingress – 504. Dlaczego?

Gateway API – następca Ingress ★

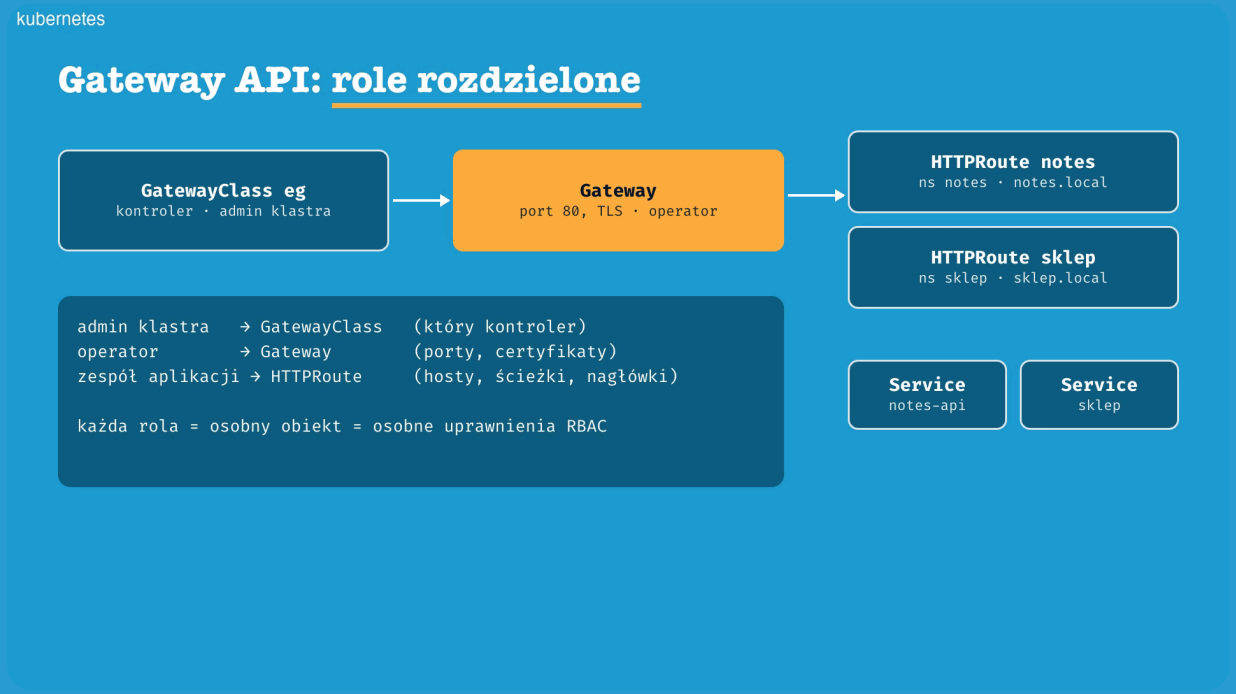
```
GatewayClass   kontroler (admin klastra)
     |
Gateway        wejście, porty, TLS (operator)
     |
HTTPRoute      host, ścieżki, nagłówki, wagi
     |           (zespół aplikacji)
Service ──► Pody
```

- **role rozdzielone, uprawnienia RBAC per obiekt**
- **wagi ruchu (canary), nagłówki, TCP/gRPC – w standardzie, bez adnotacji**

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: notes-api
  namespace: notes
spec:
  parentRefs: [{ name: notes-gateway }]
  hostnames: [notes.local]
  rules:
  - matches:
    - path: { type: PathPrefix, value: / }
    backendRefs:
    - { name: notes-api, port: 80 }
```

```
helm install eg \
  oci://docker.io/envoyproxy/gateway-helm \
  --version 1.9.3 -n envoy-gateway-system \
  --create-namespace
```

Ingress czy Gateway API



animacja: assets/animations/k8s-gateway.mp4

Ćwiczenie – notes-api w sieci klastra ★

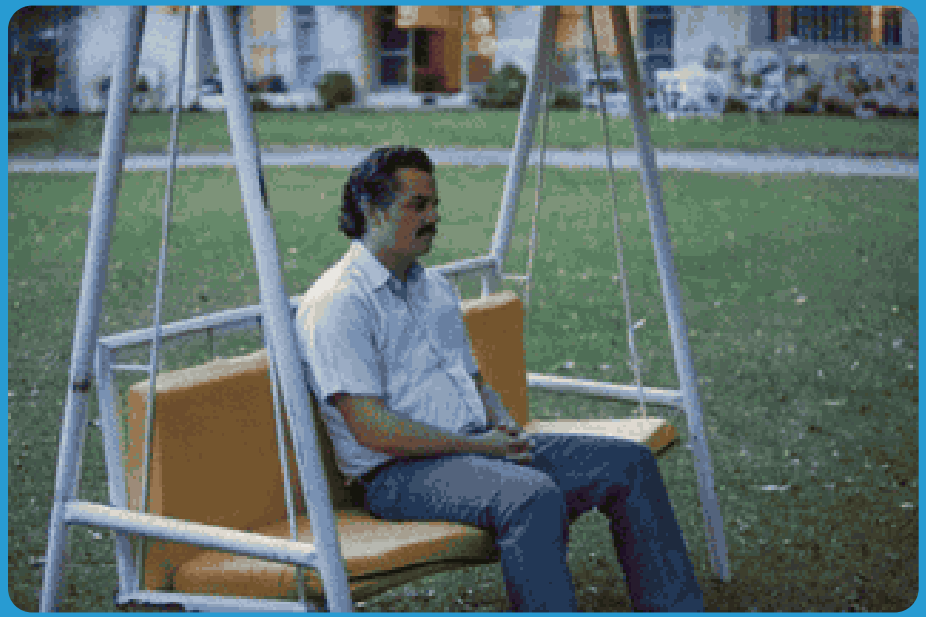
1. **Service `notes-api` (80 → 3000) i Ingress dla hosta `notes.local`**
2. **kilka zapytań przez Ingress – które pole pokazuje rozkład ruchu?**
3. **z Poda w `default`: jaka pełna nazwa DNS notes-api?**
4. **NetworkPolicy: zablokuj cały ruch w `notes`, wpuść tylko kontroler Ingress (i Envoy Gateway)**

```
curl -s -H "Host: notes.local" http://$(minikube ip)/
```

5. Storage w Kubernetesie

PersistentVolume · PersistentVolumeClaim · StorageClass · StatefulSet

Restart Poda z bazą



Dane były w `emptyDir`.

PV, PVC i StorageClass – kto za co odpowiada

Pod nie wie, skąd pochodzi dysk: prosi o PVC, a klaster dopasowuje lub tworzy PV.

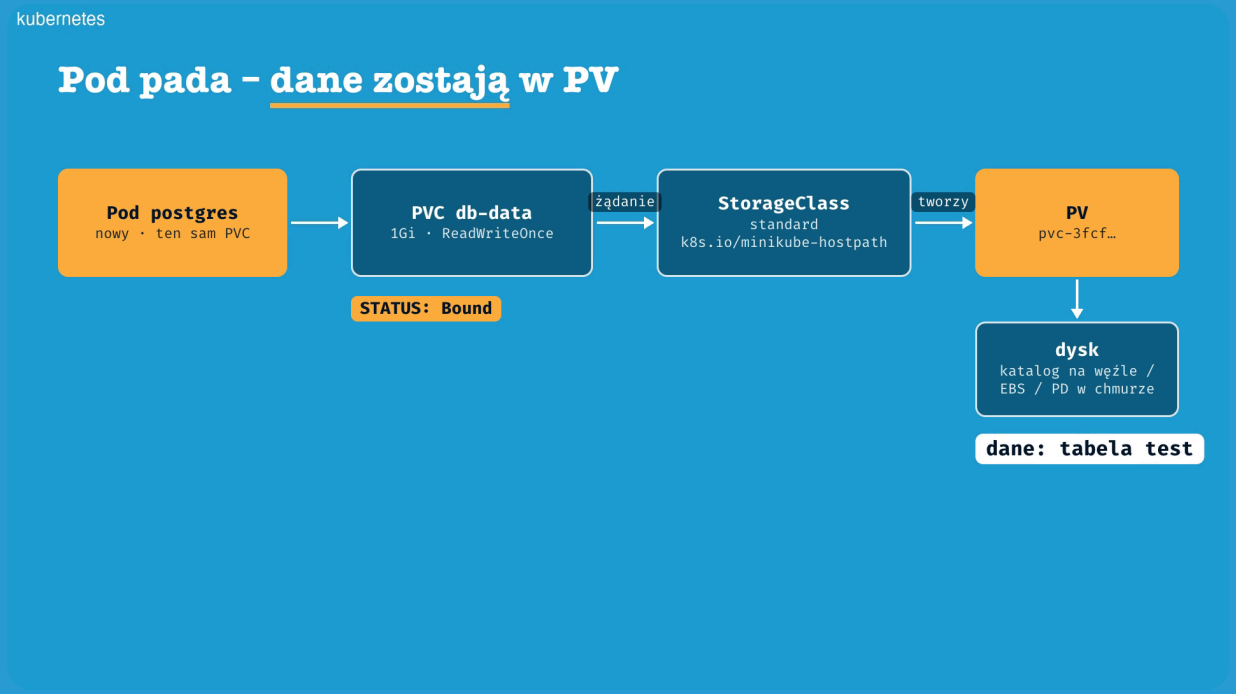
Obiekt	Czym jest	Zakres	Kto tworzy
PV – PersistentVolume	konkretny dysk: EBS, NFS, katalog na węźle	klaster	admin albo provisioner
PVC – Claim	żądanie: ile miejsca, tryb dostępu, klasa	namespace	zespół aplikacji
StorageClass	przepis na nowe PV: provisioner CSI, parametry	klaster	admin / chmura

`Pod → PVC → Bound → PV → dysk`

- **statycznie – admin zakłada PV, PVC wiąże się z pasującym**
- **dynamicznie – PVC wskazuje klasę, provisioner tworzy dysk i PV**
- `reclaimPolicy` **– co z dyskiem po usunięciu PVC**

Tryb dostępu	Zapis
RWO – ReadWriteOnce	jeden węzeł – typowy dysk
ROX – ReadOnlyMany	brak, odczyt z wielu
RWX – ReadWriteMany	wiele węzłów – NFS
RWOP – ReadWriteOncePod	jeden Pod

PV, PVC i StorageClass w akcji



animacja: assets/animations/k8s-pvc.mp4

PV i PVC – statycznie

```yaml
apiVersion: v1
kind: PersistentVolume
metadata: { name: moj-pv }
spec:
  capacity: { storage: 5Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  hostPath: { path: /data/moj-pv }
```

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: moj-pvc }
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: { storage: 2Gi }
```

```yaml
# Pod.spec
containers:
- name: app
  image: postgres:18
  volumeMounts:
  - name: db-storage
    mountPath: /var/lib/postgresql
volumes:
- name: db-storage
  persistentVolumeClaim:
    claimName: moj-pvc
```

- **PVC 2Gi wiąże się z PV 5Gi – pasuje tryb, rozmiar wystarcza**
- `Retain` **– po usunięciu PVC dane zostają, PV** `Released`

StorageClass – dyski na żądanie

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters: { type: gp3, encrypted: "true" }
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

Środowisko	Provisioner
minikube	`k8s.io/minikube-hostpath`
k3s, kind	`rancher.io/local-path`
AWS	`ebs.csi.aws.com`
GCP	`pd.csi.storage.gke.io`
Azure	`disk.csi.azure.com`

- `Delete` – **PV i dysk znikają z PVC;** `Retain` – **zostają (** `Released` **)**
- `WaitForFirstConsumer` – **dysk w strefie, w której wyląduje Pod**

Ćwiczenie 5.3 – zarządzanie danymi

```
kubectl get storageclass
kubectl apply -f persistent-volume-claim.yaml && kubectl get pvc
kubectl apply -f postgres-statefulset.yaml
kubectl rollout status statefulset/postgres
kubectl exec -it postgres-0 -- psql -U postgres
#   CREATE TABLE test (id SERIAL, name TEXT);
#   INSERT INTO test VALUES (1, 'Dane przetrwają restart');
kubectl delete pod postgres-0
kubectl exec -it postgres-0 -- psql -U postgres -c 'SELECT * FROM test;'
```

Wynik: po restarcie Pod ma tę samą nazwę i ten sam PVC – wiersz jest w tabeli.

StatefulSet, init container, Job ★

StatefulSet

- **stałe nazwy:** `notes-db-0`, `-1`**...**
- **własny PVC na Pod (**`volumeClaimTemplates`**)**
- **headless Service → DNS na Pod**
- **start i stop po kolei**

Init container

- **działa przed kontenerami aplikacji**
- **musi zakończyć się sukcesem**
- **np.** `until pg_isready -h notes-db; do sleep 2; done`

Job · CronJob

- **zadanie do skutku (**`backoffLimit`**)**
- `restartPolicy: Never | OnFailure`
- **CronJob:** `schedule: "*/1 * * * *"`
- `kubectl create job --from=cronjob/X`

Ćwiczenie – notes-api z PostgreSQL ★

1. **Secret `notes-db` poleceniem `kubectl create secret` – hasło nie trafia do Gita**
2. **StatefulSet `notes-db` (PostgreSQL 18) + headless Service + PVC; hasło z pliku (`POSTGRES_PASSWORD_FILE`)**
3. **notes-api: `STORAGE=postgres`, zmienne `PG*`, Secret jako plik, init container `pg_isready`**
4. **notatka przez jeden Pod – czy drugi ją widzi?**
5. **Job `notes-seed` wstawia notatkę do bazy**
6. **usuń `notes-db-0` – czy dane przetrwały?**

Uwaga: NetworkPolicy z sekcji 4 blokuje też bazę – co zobaczysz w logach init containera?

6. Zaawansowane tematy

Helm · UI: Headlamp, k9s, Lens · własne zasoby (CRD)

Helm – menedżer pakietów Kubernetesa

- **chart – paczka szablonów YAML +** `values.yaml`
- **release – zainstalowana instancja chartu**
- **rewizja – każda zmiana;** `rollback` **jednym poleceniem**
- **repozytoria HTTP lub OCI (jak obrazy)**
- **Helm 4 (2025) – polecenia jak w Helm 3**

```
moja-aplikacja/
  Chart.yaml          metadane
  values.yaml         domyślne wartości
  templates/          szablony YAML
```

```
CHART=oci://ghcr.io/stefanprodan/charts/podinfo
helm show values $CHART --version 6.17.0
helm install moj-podinfo $CHART --version 6.17.0 \
  --set replicaCount=2 --set service.type=NodePort
helm list · helm status moj-podinfo
helm upgrade moj-podinfo $CHART --version 6.17.0 \
  --reuse-values --set replicaCount=3
helm history moj-podinfo
helm rollback moj-podinfo 1
helm uninstall moj-podinfo
```

Własny chart

```
helm create moja-aplikacja
helm lint ./moja-aplikacja
helm template moja-aplikacja ./moja-aplikacja \
  -f custom-values.yaml
helm install moja-aplikacja ./moja-aplikacja \
  -f custom-values.yaml
```

```
# custom-values.yaml
replicaCount: 2
image: { repository: nginx, tag: "1.29" }
service: { type: NodePort, port: 80 }
```

```
# templates/deployment.yaml (fragment)
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:\
                {{ .Values.image.tag }}"
```

- `helm template` **– podgląd bez klastra**
- **jeden chart, wiele środowisk:** `-f values-staging.yaml`
- **przykład z materiałów:** `kube-prometheus-stack` **– sekcja 9**

Ćwiczenie – chart notes-api ★

1. **obejrzyj** `notes-api-chart/values.yaml` **i** `templates/` **– co trafia do ConfigMap, a co do Deploymentu?**
2. `helm template notes notes-api-chart --set storage=postgres` **– co dochodzi?**
3. **dwa środowiska:** `notes` **w** `notes-helm` **i** `notes-staging` **z** `values-staging.yaml`
4. `helm upgrade --set replicaCount=3` → `history` → `rollback`

```
curl -H "Host: notes-helm.local" http://$(minikube ip)/
```

Zarządzanie klastrem z UI

Headlamp

projekt Kubernetes SIG UI, web; addon minikube

```
minikube addons \
  enable headlamp
kubectl -n headlamp \
  port-forward \
  svc/headlamp 8080:80
```

k9s

UI w terminalu – działa w sesji SSH

`:pods` `:deploy` · `/` filtr
`l` logi · `s` shell · `d` describe

Lens / Freelens

aplikacja desktopowa, wiele klastrów

potrzebuje kubeconfig i dostępu do API z laptopa

Kubernetes Dashboard (materiały: v2.7.0) – projekt wycofany, repozytorium zarchiwizowane.

Własne zasoby – CRD

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: { name: mojeaplikacje.example.com }
spec:
  group: example.com
  scope: Namespaced
  names: { plural: mojeaplikacje, kind: MojaAplikacja, shortNames: [ma] }
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              replicas: { type: integer, minimum: 1 }
              image: { type: string }
```

- **CRD = nowy typ w API –** `kubectl get ma`**, RBAC,** `explain`
- **walidacja schematem –** `replicas: 0` **odrzuca API Server**
- **bez kontrolera nic się nie dzieje – działanie dodaje operator**
- **przykłady:** `Gateway`, `PrometheusRule`, `Certificate`, `Application` **(Argo CD)**

Ćwiczenie 6.3 – własny zasób

```
kubectl apply -f crd-mojaaplikacja.yaml
kubectl wait --for=condition=established crd/mojeaplikacje.example.com
kubectl apply -f moja-instancja.yaml
kubectl get mojeaplikacje · kubectl get ma
kubectl get mojaaplikacja pierwsza-aplikacja -o yaml
kubectl explain mojaaplikacja.spec
kubectl patch ma pierwsza-aplikacja --type=merge -p '{"spec":{"replicas":0}}'    # odrzucone
```

Wynik: obiekt istnieje, `replicas: 0` – błąd walidacji, żaden Pod nie powstaje.

7. Klaster kubeadm na maszynach wirtualnych

1 Control Plane + 2 workery · containerd · Calico · kubeconfig na jump-boksie

kubeadm – co robimy

```
jump-box ──ssh──▶ cp       kubeadm init, Calico
         ──ssh──▶ worker1  kubeadm join
         ──ssh──▶ worker2  kubeadm join
```

Na każdej VM (`common.sh`):

- **swap off, moduły `overlay`, `br_netfilter`, sysctl**
- **containerd z cgroup driver `systemd`**
- **`kubelet`, `kubeadm`, `kubectl` z `pkgs.k8s.io` (1.37)**

Krok	Gdzie	Polecenie
1	cp	`./install_master.sh` (`kubeadm init`)
2	workery	`./install_worker_nodes.sh` + `kubeadm join`
3	cp	Calico (operator), `kubectl get nodes -w`
4	cp	`./deploy_nginx.sh` + `curl <IP>:<NodePort>`
5	jump-box	kubeconfig → kontekst `kubeadm`

Poprawki w skryptach z materiałów

Problem	Poprawka
Błąd 1: brak `/etc/apt/keyrings`	`sudo mkdir -p /etc/apt/keyrings`
Błąd 2: zapis klucza GPG bez `sudo`	`curl … | gpg --dearmor | sudo tee …`
Błąd 3: Calico z manifestu v3.25.0	operator Tigera, **Calico v3.33.0** (materiały: v3.28.0)
`deploy_nginx.sh` nie czeka na Pody	`kubectl rollout status`
pula Podów `192.168.0.0/16` ★	`10.244.0.0/16` – sieci VM to często 192.168.x.0/24
`containerd` 1.7 z Ubuntu ★	**containerd 2.x** z repozytorium Dockera (`containerd.io`)

Ćwiczenie 7 – klaster kubeadm

```
# jump-box: adresy w z-jump-boxa.sh (CP, WORKERS, U)
for h in $CP $WORKERS; do ssh-copy-id "$U@$h"; scp *.sh "$U@$h:"; done
ssh -t "$U@$CP" ./install_master.sh
JOIN=$(ssh "$U@$CP" cat info_join_worker)
for h in $WORKERS; do ssh -t "$U@$h" ./install_worker_nodes.sh && ssh -t "$U@$h" "sudo $JOIN"; done
scp "$U@$CP:.kube/config" ~/.kube/kubeadm.conf     # + merge, rename-context kubeadm
kubectl --context kubeadm get nodes -o wide
```

Wynik: 3 węzły `Ready`, `calico-node` na każdym, NodePort odpowiada na każdym węźle.

8. RBAC – kontrola dostępu

Role · ClusterRole · RoleBinding · ServiceAccount · auth can-i

kubectl get pods




```
Error from server (Forbidden): pods is forbidden
```

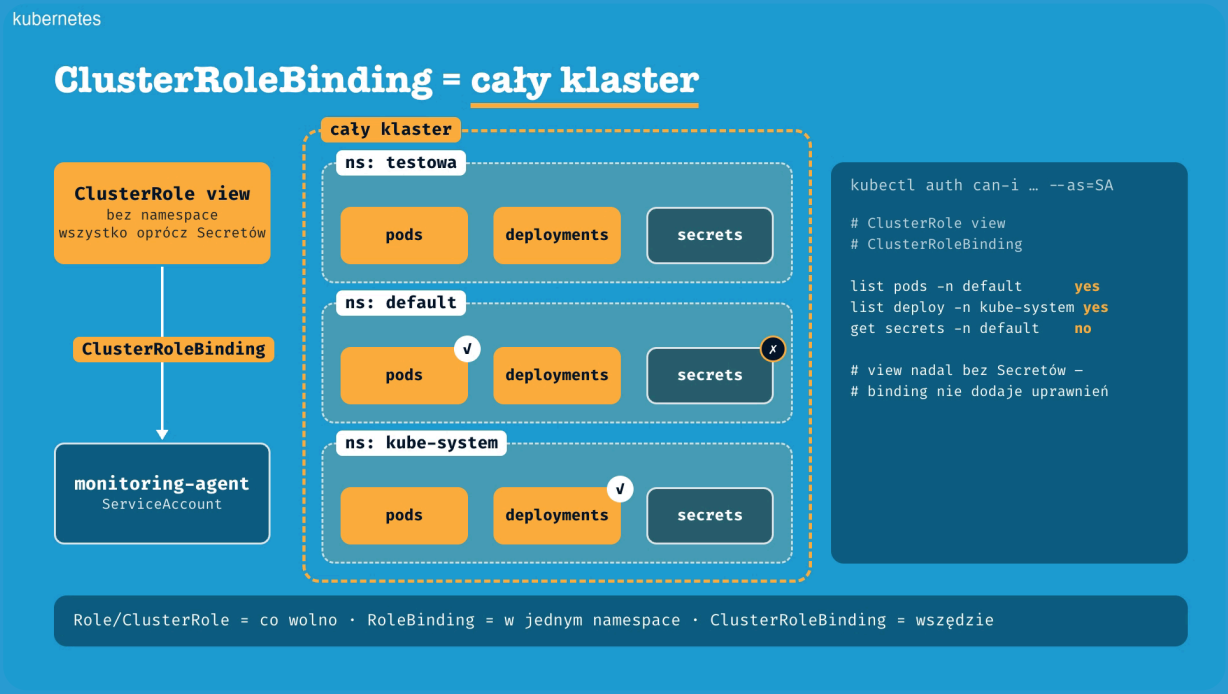
Obiekty RBAC

```
kto           ServiceAccount, User, Group
  │
RoleBinding   (namespace)
  │           ClusterRoleBinding (klaster)
Role          (namespace) · ClusterRole
  │
co            verbs × resources × apiGroups
```

- **domyślnie nic nie wolno – uprawnienia tylko dodajemy**
- **RoleBinding do ClusterRole = gotowa rola w jednym namespace**
- **wbudowane:** `view`, `edit`, `admin`, `cluster-admin`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: testowa, name: pod-reader }
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: testowa }
subjects:
- kind: ServiceAccount
  name: monitoring-agent
  namespace: testowa
roleRef:
  { kind: Role, name: pod-reader,
    apiGroup: rbac.authorization.k8s.io }
```

RBAC w akcji



animacja: assets/animations/k8s-rbac.mp4

Ćwiczenie 8.2 – tylko odczyt Podów

```
kubectl apply -f rbac-pod-reader.yaml
SA=system:serviceaccount:testowa:monitoring-agent
kubectl auth can-i list pods --as=$SA -n testowa              # yes
kubectl auth can-i delete deployments --as=$SA -n testowa     # no
kubectl auth can-i list pods --as=$SA -n default              # no
kubectl auth can-i --list --as=$SA -n testowa
AGENT='{"spec":{"serviceAccountName":"monitoring-agent"}}'
kubectl -n testowa run agent --image=registry.k8s.io/kubectl:v1.37.0 \
  --overrides="$AGENT" --rm -i --restart=Never -- get secrets
```

Wynik: Pod na koncie `monitoring-agent` listuje Pody, `get secrets` → `Forbidden`.

Ćwiczenie – konto dla CI ★

Konto `ci-deployer` w namespace `notes`:

- **aktualizuje istniejące Deploymenty (`set image`, `rollout undo`) i czeka na rollout**
- **czyta Pody i ich logi**
- **nie tworzy Deploymentów, nie usuwa Podów, nie czyta Secretów**

```
SA=system:serviceaccount:notes:ci-deployer
kubectl auth can-i patch deployments --as=$SA -n notes
kubectl --as=$SA -n notes set image deploy/notes-api api=notes-api:1.0.0
```

Ćwiczenie – logowanie kontem ServiceAccount ★

Konto `deweloper` z rolą `edit` w `zespol-a` i osobny plik kubeconfig z jego tokenem:

```
TOKEN=$(kubectl -n zespol-a create token deweloper --duration=8h)
K="kubectl config --kubeconfig=deweloper.kubeconfig"
$K set-cluster kurs --server=$SERVER --certificate-authority=ca.crt --embed-certs
$K set-credentials deweloper --token=$TOKEN
$K set-context deweloper@kurs --cluster=kurs --user=deweloper --namespace=zespol-a
$K use-context deweloper@kurs
KUBECONFIG=deweloper.kubeconfig kubectl auth whoami
```

Wynik: `system:serviceaccount:zespol-a:deweloper`**; Pody w** `zespol-a` **– tak,** `default` **i** `nodes` **–** `Forbidden`**.**

9. Monitorowanie i observability

metryki: Prometheus · wizualizacja: Grafana · logi: Loki · alerty

Trzy filary observability

Metryki

Prometheus zbiera liczby co kilkanaście sekund (pull), PromQL

`kube-state-metrics`, `node-exporter`, **kubelet/cAdvisor**

Logi

Loki – indeksuje tylko etykiety, LogQL; lżejszy od EFK (Elasticsearch + Fluent Bit + Kibana)

zbieranie: Grafana Alloy

Śledzenie

żądanie przez wiele usług: Jaeger, Tempo, OpenTelemetry

(poza zakresem ćwiczeń)

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace -f values-grafana.yaml
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring   # admin / admin123
```

Logi: Loki + Alloy

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo add grafana-community \
  https://grafana-community.github.io/helm-charts
helm install loki grafana-community/loki \
  -n monitoring -f values-loki.yaml
helm install alloy grafana/alloy \
  -n monitoring -f values-alloy.yaml
```

Grafana → Explore → Loki:

```
{namespace="notes", app="notes-api"}
{namespace="notes"} |= "POST"
sum by (pod) (
  count_over_time({namespace="notes"}[5m]))
```

Materiały: `grafana/loki-stack` **– wycofany (Loki 2.9 + Promtail bez wsparcia od 2026). Chart Loki utrzymuje teraz społeczność (grafana-community), logi zbiera Alloy.**

Alerty – PrometheusRule ★

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: notes-api
  namespace: monitoring
  labels: { release: monitoring }
spec:
  groups:
  - name: notes-api
    rules:
    - alert: NotesApiNiedostepneRepliki
      expr: |
        kube_deployment_status_replicas_unavailable{
          namespace="notes", deployment="notes-api"} > 0
      for: 1m
      labels: { severity: warning }
```

```
kubectl apply -f alert-notes-api.yaml
kubectl -n notes set image deploy/notes-api \
  api=notes-api:9.9.9
# ~2 min: Prometheus → Alerts: Firing
kubectl -n notes rollout undo deploy/notes-api
```

- `Pending` **przez** `for: 1m`**, potem** `Firing`
- **Alertmanager: grupowanie, wyciszanie, Slack, e-mail**
- **etykieta** `release: monitoring` **– inaczej Prometheus nie wczyta reguły**

Ćwiczenie 9 – metryki i logi

1. **Grafana: dashboard Kubernetes / Compute Resources / Namespace (Pods) dla** `notes`
2. **Prometheus (**`port-forward … 9090`**):**

   `kube_deployment_status_replicas_available{namespace="notes"}`
3. **Loki + Alloy: logi notes-api, odfiltruj** `POST /notes`
4. **★ alert przy obrazie** `notes-api:9.9.9` **– po jakim czasie** `Firing` **?**

```
ssh -L 3000:localhost:3000 -L 9090:localhost:9090 <użytkownik>@<jump-box>   # na laptopie
```

10. Troubleshooting

Pending · CrashLoopBackOff · OOMKilled · węzeł NotReady · sieć

`CrashLoopBackOff`, restarty: 47



Schemat diagnozy

Objaw	Pierwsze polecenie	Typowa przyczyna
`Pending`	`describe pod` → Events	brak zasobów, PVC bez dysku, nodeSelector/taint
`ErrImagePull` / `ImagePullBackOff`	`describe pod` → Events	literówka w obrazie/tagu, prywatny rejestr
`CrashLoopBackOff`	`logs X --previous`	błąd aplikacji, brak zmiennej, zły `command`
`OOMKilled` (exit 137)	`describe pod` → Last State	limit pamięci za niski
`Running`, ale `0/1`	`describe pod` → Readiness	sonda readiness nie przechodzi
Serwis milczy	`get endpointslices`	selektor ≠ etykiety, zły `targetPort`
węzeł `NotReady`	`describe node` → Conditions	kubelet, CNI, dysk/pamięć (`journalctl -u kubelet`)

Narzędzia diagnostyczne

```
kubectl get pods -A --field-selector=status.phase≠Running
kubectl get events --sort-by=.lastTimestamp -n <ns>
kubectl describe pod <pod> · kubectl logs <pod> --previous
kubectl top nodes · kubectl top pods -A --sort-by=memory
kubectl run netdebug --image=nicolaka/netshoot:v0.15 --rm -it -- bash   # nslookup, curl, traceroute
kubectl debug -it <pod> --image=busybox:1.37 --target=<kontener>        # kontener w działającym Podzie
kubectl get --raw='/readyz?verbose'
kubectl cluster-info dump --output-directory=/tmp/k8s-dump
```

Ćwiczenie – sześć awarii ★

```
kubectl create namespace awarie
kubectl apply -f scenariusze/
kubectl -n awarie get pods,pvc
```

Aplikacja	Objaw
`web`	Pody działają, Serwis nie odpowiada
`api`	nowa wersja nie startuje
`worker`	startuje i pada
`raport`	pada, choć w logach nie ma błędu
`analityka`	czeka i czeka
`archiwum`	Pod nie dostaje dysku

Diagnoza tylko `get`, `describe`, `logs`, `events` – bez zaglądania do YAML.

11. AI w pracy z Kubernetesem

manifesty od AI · weryfikacja · k8sgpt · agenci z uprawnieniami

Wdrożyłem manifest od AI bez czytania



Pod ma `privileged: true` **i obraz** `:latest`**.**

Manifest od AI – co może pójść źle ★

```yaml
apiVersion: extensions/v1beta1   # usunięte w 1.16
kind: Deployment
spec:
  replicas: 3                    # brak selector
  template:
    metadata: { labels: { app: notes } }
    spec:
      containers:
      - image: notes-api:latest
        imagePullPolicy: Always
        ports: [{ containerPort: 8080 }]   # app: 3000
        env:
        - { name: PGPASSWORD, value: "SuperTajne" }
        livenessProbe:
          httpGet: { path: /health, port: 8080 }  # /healthz
```

Narzędzie	Znajduje
`kubeconform -strict`	stare API, złe pola
`kubectl apply --dry-run=server`	to, czego nie przyjmie klaster
`kube-linter`	`latest`, root, brak zasobów
`kubectl diff`	co zmieni się w klastrze
człowiek	**port 8080, sonda** `/health`

k8sgpt i agenci ★

```
k8sgpt analyze --namespace awarie
k8sgpt analyze --filter=Pod
# z modelem (opis błędów idzie do dostawcy):
k8sgpt auth add --backend anthropic \
  --model <model>
k8sgpt analyze --explain --anonymize
```

analizatory działają bez AI – znalazły wszystkie 6 awarii z sekcji 10

kubectl-ai i inne agenty wykonują polecenia z Twoim kubeconfigiem:

- **dawaj konto z ClusterRole `view` (sekcja 8), nie `cluster-admin`**
- **logi i manifesty to dane firmy – model zatwierdzony albo lokalny (Ollama)**
- **weryfikacja przed `apply`: kubeconform → `--dry-run=server` → `diff` → checklista**

Od Dockera do Kubernetesa – podsumowanie

Docker	Kubernetes
obraz, rejestr	te same obrazy OCI
`docker run`, `--restart`	Pod, Deployment, ReplicaSet
`HEALTHCHECK`	`readinessProbe`, `livenessProbe`
`-p`, DNS w sieci	Service, Ingress / Gateway API, CoreDNS
`--scale` + load balancer	`replicas`, HPA, Service
wolumin	PersistentVolume, PVC, StorageClass
`.env` · `secrets:`	ConfigMap · Secret
`--memory`, `--cpus`	`resources.requests` / `limits`
`compose.yaml`	manifesty YAML, Helm

Pytania?

dziękuję · materiały: kubernetes/